

ĐẠI HỌC BÁCH KHOA HÀ NỘI



ĐỒ ÁN TỐT NGHIỆP

**Xây dựng nền tảng thương mại điện tử đa thuê bao
với kiến trúc Zero-file và Dynamic Rendering**

Trịnh Vi Giang Xuân
xuan.tvg225118@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin

Giảng viên hướng dẫn: TS. Nguyễn Quốc Tuấn

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành nhất đến TS. Nguyễn Quốc Tuấn. Thầy đã tận tình hướng dẫn, định hướng phát triển và chỉ bảo cặn kẽ, giúp em hoàn thành đồ án tốt nghiệp này.

Em cũng xin trân trọng cảm ơn Ban Giám hiệu Trường Công nghệ Thông tin và Truyền thông cùng quý thầy cô trong trường. Những kiến thức nền tảng quý giá mà các thầy cô truyền đạt chính là cơ sở vững chắc để em thực hiện đề tài. Bên cạnh đó, em xin tri ân gia đình, bạn bè đã luôn động viên, hỗ trợ và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập.

Cuối cùng, em xin cam đoan các kết quả trong báo cáo là thành quả trung thực của bản thân. Dù đã nỗ lực, đồ án vẫn khó tránh khỏi thiếu sót do giới hạn về kiến thức và kinh nghiệm. Em rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh thương mại điện tử tại Việt Nam đang phát triển mạnh mẽ, các doanh nghiệp vừa và nhỏ thường xuyên đối mặt với rào cản lớn về mặt chi phí cũng như rào cản kỹ thuật khi muốn xây dựng và vận hành một cửa hàng trực tuyến độc lập. Các nền tảng thương mại điện tử hiện có trên thị trường như Shopify, WooCommerce hay Haravan vẫn tồn tại nhiều hạn chế đáng kể, đặc biệt là trong việc không thể đáp ứng đồng thời cả ba tiêu chí: chi phí khởi tạo và duy trì thấp, khả năng tùy biến giao diện linh hoạt mà không yêu cầu kiến thức lập trình, và khả năng ~~mở rộng hệ thống để phục vụ hàng chục ngàn vận hành nhiều~~ cửa hàng ~~mà không làm gia tăng tuyến tính chi phí trên cùng một~~ hạ tầng ~~dùng chung với chi phí biên thấp cho mỗi cửa hàng tăng thêm~~. Xuất phát từ thực tiễn đó, đồ án này tập trung vào việc nghiên cứu và phát triển nền tảng thương mại điện tử đa thuê bao mang tên ShopVolo v2 nhằm giải quyết triệt để bài toán tối ưu hóa nguồn lực. Hướng tiếp cận chủ đạo của nghiên cứu là áp dụng kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file", trong đó toàn bộ giao diện của các cửa hàng được biểu diễn dưới dạng dữ liệu cấu hình thay vì mã nguồn vật lý riêng biệt.

Hệ thống được phát triển dựa trên mô hình Monorepo với công cụ Turborepo, tích hợp các nền tảng công nghệ hiện đại bao gồm Next.js cho giao diện trang quản trị và kết xuất mặt tiền cửa hàng động, NestJS cho hệ thống API lõi, cùng hệ thống cơ sở dữ liệu kép kết hợp giữa PostgreSQL và MongoDB để xử lý hiệu quả cả thông tin có cấu trúc lẫn tài liệu cấu hình linh hoạt. Đồ án mang lại ba đóng góp kỹ thuật chính: (i) cơ chế cô lập dữ liệu đa thuê bao tự động dựa trên AsyncLocalStorage, bảo đảm mọi truy vấn gắn đúng phạm vi cửa hàng mà không cần truyền tham số qua các tầng xử lý; (ii) động cơ kết xuất giao diện động theo kiến trúc Zero-file, cho phép thêm cửa hàng mới mà không phát sinh tệp mã nguồn hay lần biên dịch nào; và (iii) mô hình Master Template theo ngành hàng kết hợp quy trình chỉnh sửa hai pha nháp–xuất bản, giúp người không biết lập trình tùy biến giao diện một cách an toàn. Kết quả của đồ án là một hệ thống hoàn chỉnh gồm bốn ứng dụng và năm gói thư viện dùng chung, hiện thực hóa đầy đủ mười nhóm chức năng đã đề ra và được kiểm chứng bằng bộ kiểm thử tự động phủ các nghiệp vụ cốt lõi.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the context of the rapidly growing e-commerce sector in Vietnam, small and medium-sized enterprises (SMEs) frequently encounter significant financial and technical barriers when attempting to build and operate independent online stores. Existing e-commerce platforms on the market, such as Shopify, WooCommerce, or Haravan, still exhibit considerable limitations. Specifically, they fail to simultaneously satisfy three crucial criteria: low initial and maintenance costs, flexible interface customization without requiring programming knowledge, and ~~system scalability capable of serving tens of thousands of stores without linearly increasing infrastructure costs~~the ability to operate many stores on a shared infrastructure with a low marginal cost for each additional store. Stemming from this practical need, this thesis focuses on the research and development of a multi-tenant e-commerce platform named ShopVolo v2 to thoroughly address the resource optimization problem. The primary approach of the study involves implementing a Multi-tenant Software as a Service (SaaS) architecture combined with the "Zero-file" design principle, wherein the entire storefront interface is represented as configuration data rather than discrete physical source code files.

The system is developed based on a Monorepo model using the Turborepo tool, integrating modern technology stacks including Next.js for the administrative dashboard interface and dynamic storefront rendering, and NestJS for the core API system. Furthermore, a dual database system combining PostgreSQL and MongoDB is employed to efficiently process both structured information and flexible configuration documents. The thesis provides three main technical contributions: (i) an automatic multi-tenant data isolation mechanism based on AsyncLocalStorage, ensuring that every query is bound to the correct store scope without passing parameters through processing layers; (ii) a dynamic rendering engine following the Zero-file architecture, allowing new stores to be added without generating any source files or rebuilds; and (iii) an industry-based Master Template model combined with a two-phase draft-publish editing workflow, enabling non-programmers to customize their storefronts safely. The result is a complete system consisting of four applications and five shared library packages, fully implementing the ten proposed functional groups and verified by an automated test suite covering the core business flows.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	5
1.4 Bố cục đồ án	6
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	8
2.1 Khảo sát hiện trạng	8
2.2 Tổng quan chức năng	10
2.2.1 Biểu đồ use case tổng quát	11
2.2.2 Biểu đồ use case phân rã — Quản lý cửa hàng	12
2.2.3 Biểu đồ use case phân rã — Sản phẩm và Đơn hàng.....	13
2.2.4 Biểu đồ use case phân rã — Trải nghiệm khách hàng	14
2.2.5 Quy trình nghiệp vụ — Khởi tạo cửa hàng và kết xuất giao diện.....	14
2.3 Đặc tả chức năng	14
2.3.1 Đặc tả use case: Khởi tạo cửa hàng (UC-01)	16
2.3.2 Đặc tả use case: Tùy biến giao diện và thương hiệu (UC-03)	17
2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04)	18
2.3.4 Đặc tả use case: Duyệt sản phẩm trên trang cửa hàng (UC-07)	20
2.3.5 Đặc tả use case: Thanh toán và đặt đơn hàng (UC-09)	21
2.4 Yêu cầu phi chức năng	21
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	25
3.1 Turborepo — Quản lý Monorepo	25
3.2 NestJS — Framework Backend.....	26
3.3 Next.js — Framework Frontend.....	26

3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ	27
3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu	28
3.6 Redis — Cache In-memory — Bộ nhớ đệm trong RAM	28
3.7 MinIO — Object Storage — Lưu trữ đối tượng	29
3.8 Better Auth — Xác thực và Phân quyền	29
3.9 Docker và Kubernetes — Containerization — Đóng gói và Orchestration điều phối	30
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	33
4.1 Thiết kế kiến trúc	33
4.1.1 Lựa chọn kiến trúc phần mềm	33
4.1.2 Thiết kế tổng quan	34
4.1.3 Thiết kế chi tiết gói api-core	34
4.2 Thiết kế chi tiết	35
4.2.1 Thiết kế giao diện	35
4.2.2 Thiết kế lớp	36
4.2.3 Thiết kế cơ sở dữ liệu	36
4.3 Xây dựng ứng dụng	42
4.3.1 Thư viện và công cụ sử dụng	42
4.3.2 Kết quả đạt được	42
4.3.3 Minh họa các chức năng chính	43
4.4 Kiểm thử	43
4.5 Triển khai	44
4.5.1 <u>Đánh giá hiệu năng (benchmark)</u>	45
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	47
5.1 Cơ chế cô lập dữ liệu đa thuê bao dựa trên AsyncLocalStorage	47
5.1.1 Bài toán	47

5.1.2 Giải pháp	48
5.1.3 Kết quả đạt được	49
5.2 Kiến trúc Zero-file và động cơ kết xuất giao diện động	51
5.2.1 Bài toán	51
5.2.2 Giải pháp	51
5.2.3 Kết quả đạt được	53
5.3 Mô hình Master Template và quy trình cá nhân hóa giao diện hai pha.....	54
5.3.1 Bài toán	54
5.3.2 Giải pháp	54
5.3.3 Kết quả đạt được	56
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	58
6.1 Kết luận	58
6.2 Hướng phát triển.....	59
TÀI LIỆU THAM KHẢO.....	62
PHỤ LỤC.....	64
A. ĐẶC TẢ USE CASE BỔ SUNG.....	64
A.1 Đặc tả use case “Quản lý khuyến mãi” (UC-02).....	64
A.2 Đặc tả use case “Quản lý giỏ hàng” (UC-08)	65
A.3 Đặc tả use case “Ánh xạ tên miền tùy chỉnh” (UC-10)	66

DANH MỤC HÌNH VẼ

Hình 1.1	<u>So sánh-Thị phần các nền tảng trong nhóm một triệu website thương mại điện tử lớn nhất (đơn vị: %, số liệu năm 2025–2026) [6], [7], [8]</u>	3
Hình 1.2	<u>Doanh thu thương mại điện tử bán lẻ B2C của Việt Nam giai đoạn 2020–2024 (đơn vị: tỷ USD) [13], [14]</u>	4
Hình 2.1	Biểu đồ use case tổng quát hệ thống ShopVolo v2	11
Hình 2.2	Biểu đồ use case phân rã nhóm Quản lý cửa hàng	12
Hình 2.3	Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng	13
Hình 2.4	Biểu đồ use case phân rã nhóm Trải nghiệm khách hàng	14
Hình 2.5	Biểu đồ hoạt động quy trình khởi tạo cửa hàng và kết xuất giao diện động	15
Hình 2.6	Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm	19
Hình 2.7	Biểu đồ hoạt động quy trình thanh toán và đặt đơn hàng	22
Hình 4.1	Biểu đồ gói thể hiện kiến trúc phân tầng và quan hệ phụ thuộc của hệ thống	34
Hình 4.2	Thiết kế chi tiết bên trong gói api-core và luồng xử lý đa thuê bao	35
Hình 4.3	Biểu đồ lớp các thành phần cốt lõi của api-core	37
Hình 4.4	Biểu đồ trình tự luồng đặt hàng (checkout)	38
Hình 4.5	Biểu đồ trình tự luồng xác thực qua BetterAuthGuard	39
Hình 4.6	Sơ đồ thực thể liên kết các thực thể cốt lõi trên PostgreSQL	40
Hình 4.7	<u>Thiết kế tài liệu các bộ sưu tập giao diện trên MongoDB</u>	41
Hình 4.8	<u>Biểu đồ kết quả đo hiệu năng theo số lượng cửa hàng</u>	46
Hình 5.1	Biểu đồ trình tự quá trình định danh tenant và truyền ngữ cảnh qua AsyncLocalStorage	50
Hình 5.2	Biểu đồ hoạt động luồng kết xuất giao diện động theo kiến trúc Zero-file	53
Hình 5.3	<u>Biểu đồ hoạt động quy trình chỉnh sửa nháp và xuất bản giao diện hai pha</u>	55
Hình 5.4	Mô hình hợp nhất cấu hình giao diện phân tầng và chu trình nháp–xuất bản	56

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh chi tiết các nền tảng thương mại điện tử	9
Bảng 2.2	Đặc tả use case Khởi tạo cửa hàng (UC-01)	16
Bảng 2.3	Đặc tả use case Tùy biến giao diện và thương hiệu (UC-03) . .	17
Bảng 2.4	Đặc tả use case Quản lý sản phẩm (UC-04)	18
Bảng 2.5	Đặc tả use case Duyệt sản phẩm trên trang cửa hàng (UC-07)	20
Bảng 2.6	Đặc tả use case Thanh toán và đặt đơn hàng (UC-09)	21
Bảng 3.1	Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2 . .	31
Bảng 4.1	Danh sách thư viện và công cụ sử dụng trong dự án	42
Bảng 4.2	Thống kê thông số kỹ thuật của dự án	42
Bảng 4.3	Các trường hợp kiểm thử tiêu biểu cho luồng đặt hàng	44
Bảng A.1	Đặc tả use case Quản lý khuyến mãi (UC-02)	64
Bảng A.2	Đặc tả use case Quản lý giỏ hàng (UC-08)	65
Bảng A.3	Đặc tả use case Ảnh xạ tên miền tùy chỉnh (UC-10)	66

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ACID	Tính nguyên tử, nhất quán, độc lập và bền vững (Atomicity, Consistency, Isolation, Durability)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CI/CD	Tích hợp liên tục và Triển khai liên tục (Continuous Integration/Continuous Deployment)
COD	Thanh toán khi nhận hàng (Cash On Delivery)
CRUD	Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)
DAG	Đồ thị có hướng phi chu trình (Directed Acyclic Graph)
DNS	Hệ thống phân giải tên miền (Domain Name System)
DTO	Đối tượng truyền dữ liệu (Data Transfer Object)
ER	Thực thể - Kết hợp (Entity-Relationship)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
JSON	JavaScript Object Notation
ODM	Ánh xạ tài liệu đối tượng (Object-Document Mapping)
ORM	Ánh xạ quan hệ đối tượng (Object-Relational Mapping)
REST	Representational State Transfer
RSC	Thành phần kết xuất phía máy chủ của React (React Server Components)
S3	Dịch vụ lưu trữ đơn giản (Simple Storage Service)

Thuật ngữ	Ý nghĩa
SaaS	Software as a Service — Phần mềm dịch vụ
SKU	Mã định danh đơn vị lưu kho (Stock Keeping Unit)
SME	Doanh nghiệp vừa và nhỏ (Small and Medium Enterprise)
SSR	Kết xuất phía máy chủ (Server-Side Rendering)
TTL	Thời gian sống (Time To Live)
UC	Trường hợp sử dụng (Use Case)
UI	Giao diện người dùng (User Interface)
UML	Ngôn ngữ mô hình hóa thống nhất (Unified Modeling Language)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu tổng quan về bối cảnh và mục tiêu của đề tài nghiên cứu. Đầu tiên, phần 1.1 đặt vấn đề từ thực tiễn nhu cầu xây dựng cửa hàng trực tuyến của các doanh nghiệp vừa và nhỏ tại Việt Nam. Tiếp đó, phần 1.2 phân tích các giải pháp hiện có trên thị trường để làm rõ những hạn chế và xác định các mục tiêu cụ thể của hệ thống đề xuất. Phần 1.3 trình bày ngắn gọn định hướng kiến trúc đa thuê bao và công nghệ được lựa chọn để giải quyết bài toán. Cuối cùng, phần 1.4 mô tả bố cục chi tiết của toàn bộ báo cáo đồ án tốt nghiệp này.

1.1 Đặt vấn đề

Thương mại điện tử tại Việt Nam đang trải qua giai đoạn tăng trưởng mạnh mẽ trong những năm gần đây [1]. Sự chuyển dịch thói quen mua sắm của người tiêu dùng sang các nền tảng trực tuyến đã thúc đẩy các hộ kinh doanh và doanh nghiệp vừa và nhỏ liên tục tìm kiếm cơ hội mở rộng kênh bán hàng. Nhu cầu sở hữu một cửa hàng trực tuyến riêng biệt, chuyên nghiệp để xây dựng thương hiệu đang ngày càng trở nên cấp thiết.

Tuy nhiên, để ~~xây dựng và vận hành sở hữu~~ một cửa hàng trực tuyến độc lập, các doanh nghiệp vừa và nhỏ ~~phải đối mặt với nhiều~~ hiện đứng trước hai lựa chọn, mỗi lựa chọn đều kèm theo rào cản lớn. Chi riêng:

- Thuê bao một nền tảng dịch vụ (SaaS): chi phí ~~ban đầu cho việc thiết kế và phát triển website thường định kỳ~~ dao động từ ~~mười~~ khoảng 300.000 đồng đến năm mươi triệu hơn 1.500.000 đồng mỗi tháng với nền tảng nội địa Haravan [2], hoặc từ 39 USD mỗi tháng trở lên với nền tảng quốc tế Shopify [3], [4], chưa tính chi phí mua giao diện và tiện ích mở rộng. Bên cạnh đó, việc duy trì một đội ngũ kỹ thuật để quản lý
- Tự triển khai một giải pháp mã nguồn mở như WooCommerce: phần lõi miễn phí nhưng doanh nghiệp phải tự thuê máy chủ ~~,(khoảng 10–100 USD mỗi tháng)~~ và tự chịu trách nhiệm về bảo mật, sao lưu và cập nhật ~~tính năng liên tục là~~; chi phí xây dựng một cửa hàng tùy biến qua đơn vị phát triển có thể vượt 8.000 USD [5].

Cả hai lựa chọn đều đặt ra gánh nặng ~~tài chính không nhỏ. Hơn nữa, về chi phí hoặc về năng lực kỹ thuật, đồng thời~~ gian triển khai từ khi lên ý tưởng đến khi ra mắt thường kéo dài từ thời gian thiết lập một đến ba tháng cửa hàng hoàn chỉnh cũng đáng kể, làm chậm khả năng phản ứng với thị trường.

Các nền tảng phần mềm dưới dạng dịch vụ hiện có trên thị trường vẫn tồn tại

nhiều hạn chế đáng kể. Các nền tảng quốc tế thường có chi phí duy trì hàng tháng cao, không tối ưu cho các phương thức thanh toán địa phương và giới hạn khả năng can thiệp sâu. Các giải pháp tự triển khai lại đòi hỏi người sử dụng phải có kiến thức kỹ thuật chuyên sâu để tự thiết lập hạ tầng và bảo mật. Trong khi đó, các nền tảng trong nước tuy dễ tiếp cận nhưng lại thiếu đi khả năng cho phép người dùng tự do tùy biến giao diện một cách sâu rộng mà không cần viết mã. Hiện tại, chưa có giải pháp nào đáp ứng đồng thời cả ba tiêu chí: chi phí thấp, khả năng tùy biến cao, và khả năng vận hành ~~hàng chục ngàn~~ hàng cửa hàng ~~mà không làm tăng tuyến tính chi phí trên cùng một~~ hạ tầng dùng chung với chi phí biên thấp cho mỗi cửa hàng tăng thêm.

Một nền tảng phần mềm mở, cho phép các doanh nghiệp vừa và nhỏ tại Việt Nam khởi động cửa hàng trực tuyến chỉ trong vài phút mà không cần kỹ năng lập trình sẽ là một bước tiến quan trọng. Giải pháp này không chỉ giúp tiết kiệm chi phí và thời gian triển khai, mà còn cung cấp một giao diện chuyên nghiệp, góp phần đẩy nhanh quá trình số hóa thương mại.

1.2 Mục tiêu và phạm vi đề tài

Trên thị trường hiện nay, có nhiều nền tảng cung cấp giải pháp xây dựng cửa hàng trực tuyến với các đặc điểm khác nhau. ~~Shopify~~:

- ~~Shopify~~ là ~~một trong những~~ nền tảng ~~cung cấp phần mềm dưới dạng dịch vụ~~ hàng SaaS dẫn đầu thị trường, chiếm khoảng 28,8% thị phần trong nhóm một triệu website thương mại điện tử lớn nhất thế giới [6], [7]. Nền tảng cung cấp giao diện đẹp ~~mắt và~~ hạ tầng ổn định, nhưng ~~có~~ chi phí ~~duy trì khá cao từ \$29 đến \$299 thuê bao cao (gói Basic 39 USD, Advanced 399 USD mỗi tháng [3]. Hơn nữa, nền tảng này cũng) [3], [4],~~ giới hạn khả năng tùy biến sâu vào mã nguồn giao diện ~~, và chưa thực sự phù hợp với các doanh nghiệp nhỏ tại Việt Nam về mặt chi phí và hỗ trợ tối ưu cho~~ thanh toán nội địa ~~.~~

~~WooCommerce Việt Nam.~~

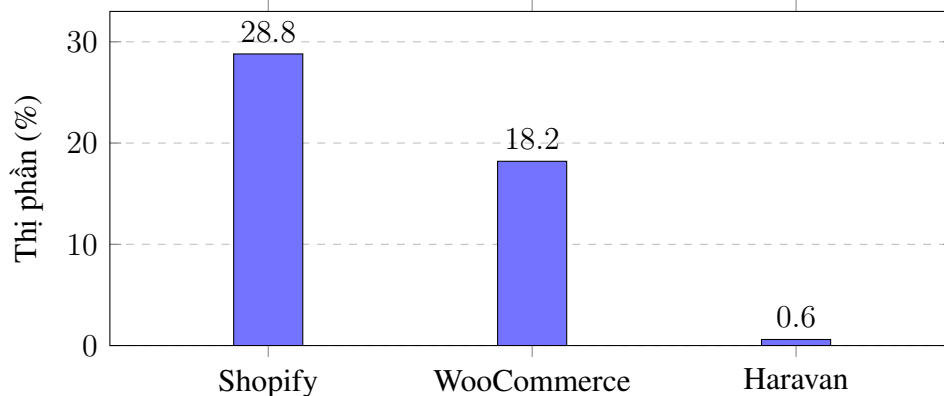
- ~~WooCommerce~~ là ~~một~~ thành phần ~~mở rộng~~ mã nguồn mở ~~dành cho hệ thống quản trị nội dung WordPress, cho phép tạo chạy trên WordPress, chiếm khoảng 18,2% nhóm một triệu website lớn nhất với hơn 4,5 triệu cửa hàng trực tuyến hoàn toàn đang hoạt động [8], [9]. Phần lõi miễn phí [10]. Mặc dù mang lại khả năng và~~ tùy biến sâu, ~~giải pháp này lại yêu cầu nhưng~~ người sử dụng ~~dùng~~ phải tự thuê máy chủ ~~lưu trữ~~, tự cấu hình bảo mật và ~~thường xuyên thực hiện các bản cập nhật . Việc này —~~ đòi hỏi trình độ kỹ thuật cao ~~và tiêu tốn nhiều thời gian vận hành hệ thống.~~

Tại Việt Nam, Haravan.

- **Haravan** là một nền tảng phổ biến tại Việt Nam với hơn 50.000 doanh nghiệp đăng ký, thân thiện với người dùng trong nước và tích hợp tốt các dịch vụ vận chuyển, thanh toán nội địa [11][2], [11]. Tuy nhiên, giải pháp này nền tảng vẫn có những hạn chế nhất định trong việc khả năng tùy biến giao diện nếu người dùng không biết lập trình, đồng thời hệ thống và chưa được xây dựng dựa trên kiến trúc đa thuê bao thực sự tối ưu cho việc cá nhân hóa giao diện động. Đối với các doanh nghiệp lớn, Magento là

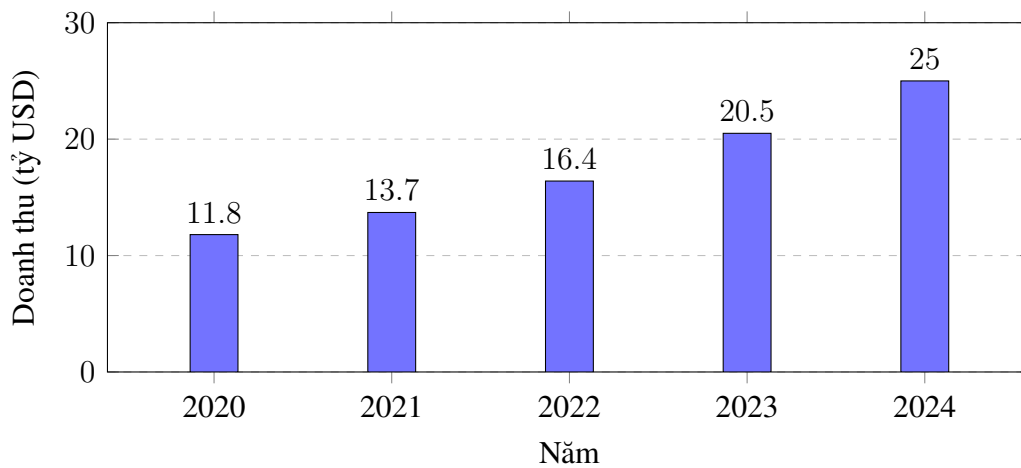
Hình 1.1 so sánh thị phần của ba nền tảng tiêu biểu trong nhóm một lựa chọn mạnh mẽ, nhưng đi kèm với sự phức tạp và chi phí triển khai cực kỳ đắt đỏ [12] triệu website thương mại điện tử lớn nhất, cho thấy thị trường bị chi phối bởi các nền tảng quốc tế, trong khi nền tảng nội địa còn chiếm tỷ trọng rất nhỏ ở phạm vi toàn cầu.

Tiêu chí Shopify WooCommerce Haravan ShopVolo v2
Chi phí thấp × ✓
Trung bình ✓ Không cần lập trình × × ✓ ✓ Tùy biến giao diện sâu Hạn chế ✓
(cần dev) × ✓ Multi-tenant thực sự × × × ✓ Phù hợp SME Việt Nam × Khó ✓
✓ Vận hành 20k+ shop / hạ tầng nhỏ N/A × N/A ✓



Hình 1.1: So sánh Thị phần các nền tảng trong nhóm một triệu website thương mại điện tử lớn nhất (đơn vị: %, số liệu năm 2025–2026) [6], [7], [8]

Bảng ?? tổng hợp sự khác biệt giữa các nền tảng



Hình 1.2: Doanh thu thương mại điện tử bán lẻ B2C của Việt Nam giai đoạn 2020–2024 (đơn vị: tỷ USD) [13], [14]

Hình 1.2 cho thấy doanh thu thương mại điện tử **hiện nay. Có thể bán lẻ B2C** của Việt Nam tăng từ 11,8 tỷ USD năm 2020 lên khoảng 25 tỷ USD năm 2024, tương ứng tốc độ tăng trưởng kép hàng năm khoảng 20% [13], [14]. Đà tăng trưởng liên tục này, cùng với việc số lượng doanh nghiệp tham gia bán hàng trực tuyến ngày càng lớn, cho thấy nhu cầu sở hữu cửa hàng trực tuyến riêng là rất cao và còn nhiều dư địa. Tuy nhiên, như đã phân tích ở trên, các nền tảng hiện có chưa giải quyết được đồng thời ba vấn đề cốt lõi: **(i)-chi-**

- **Chi** phí triển khai và duy trì thấp, **(ii)-khả-**
- **Khả** năng tùy biến giao diện sâu rộng mà không cần lập trình, **và (iii)-khả năng vận hành một số lượng lớn-**
- **Chi phí biên thấp khi phục vụ thêm** cửa hàng trên **cùng** một hạ tầng **phần cứng tối ưu, dùng chung.**

Bảng so sánh chi tiết các nền tảng theo từng tiêu chí được trình bày ở mục 2.1 (Bảng 2.1).

Đề tài hướng đến xây dựng nền tảng ShopVolo v2 nhằm giải quyết các bài toán trên với các chức năng chính bao gồm: **(i)-khởi-**

- **Khởi** tạo cửa hàng trực tuyến trong vài phút thông qua các bước hướng dẫn tự động, **(ii)-cung-**
- **Cung** cấp công cụ tùy biến giao diện trực quan không cần lập trình, **(iii)-quản**
- **Quản** lý vòng đời sản phẩm và trạng thái đơn hàng đầy đủ, **(iv)-xử-**
- **Xử** lý thanh toán trực tuyến qua các cổng thanh toán **nội địa (VNPAY, MoMo)**

hoặc hình thức nhận hàng trả tiền ~~, và (v)hỗ (COD).~~

- Hỗ trợ ánh xạ tên miền tùy chỉnh riêng biệt cho từng cửa hàng.

1.3 Định hướng giải pháp

Để ~~giải quyết bài toán vận hành số lượng lớn cửa hàng với~~ ha chi phí biên khi phục vụ thêm cửa hàng trên một ~~hạ~~ tầng ~~tối ưu~~ dùng chung, đề tài lựa chọn kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file". Đây là phương pháp quản lý giao diện trong đó toàn bộ cấu trúc trang của các cửa hàng được biểu diễn hoàn toàn dưới dạng tệp dữ liệu cấu hình JSON thay vì các tệp mã nguồn riêng lẻ. Cách tiếp cận này cho phép một mã nguồn duy nhất có thể phục vụ hàng ngàn nhiều cửa hàng khác nhau mà không cần thực hiện quá trình triển khai lại từng ứng dụng riêng biệt.

Hệ thống ShopVolo v2 được xây dựng dưới dạng Monorepo sử dụng Turborepo, bao gồm bốn ứng dụng chính. Ứng dụng Admin Dashboard phát triển bằng Next.js 16 cho phép người bán quản lý cửa hàng toàn diện. Ứng dụng API Core viết bằng NestJS 11 cung cấp các giao diện lập trình ứng dụng REST API có khả năng cô lập dữ liệu tự động qua AsyncLocalStorage. Ứng dụng Storefront cũng dùng Next.js 16 để kết xuất giao diện động từ cấu hình, cùng với một công cụ dòng lệnh hỗ trợ vận hành hệ thống hàng loạt. Cơ sở dữ liệu PostgreSQL được dùng cho các dữ liệu có cấu trúc chặt chẽ như đơn hàng, trong khi MongoDB lưu trữ các dữ liệu có tính chất linh hoạt như cấu hình giao diện.

Đóng góp của đề tài tập trung ở ba giải pháp kỹ thuật. ~~Thứ nhất là cơ chế cô lập dữ liệu đa thuê bao tự động bằng AsyncLocalStorage, giúp:~~

- Cô lập dữ liệu đa thuê bao tự động bằng AsyncLocalStorage: mọi truy vấn được gắn đúng phạm vi cửa hàng mà không cần truyền tham số qua các tầng xử lý, đưa trách nhiệm cô lập dữ liệu về một điểm duy nhất thay vì phụ thuộc kỷ luật của người lập trình. ~~Thứ hai là kiến trúc giao diện “Zero-file” với động cơ kết xuất động,~~
- Kiến trúc giao diện “Zero-file” với động cơ kết xuất động: cho phép thêm cửa hàng mới mà không sinh thêm tệp mã nguồn hay lần biên dịch nào. ~~Thứ ba:~~ chi phí biên của mỗi cửa hàng chỉ còn là mô-vài tài liệu cấu hình Master Template kết hợp quy trình nháp-xuất bản hai pha, giúp JSON.
- Mô hình Master Template kết hợp quy trình nháp-xuất bản hai pha: chủ cửa hàng có ngay giao diện hoàn chỉnh theo ngành hàng rồi tùy biến an toàn; hệ thống chỉ lưu cấu hình thay đổi của từng cửa hàng và dùng bộ nhớ đệm phân tầng (Redis cho định danh cửa hàng, Next.js Data Cache cho bố cục) để

đạt hiệu năng cao. ~~Mục~~

~~Ba giải pháp trên cùng hướng tới một mục tiêu thiết kế: hạ chi phí biên của hệ thống là mỗi cửa hàng xuống mức tối thiểu để có thể phục vụ đồng thời 20.000 nhiều cửa hàng trên một máy chủ 16GB RAM với thời gian phản hồi dưới 500ms đối với dữ liệu mới và dưới 10ms đối với dữ liệu đã có trong bộ nhớ đệm hạ tầng dùng chung mà vẫn giữ được trải nghiệm dịch vụ tốt.~~ Chi tiết về ~~các cách hiện thực hóa và phân tích từng~~ giải pháp kỹ thuật ~~này sẽ~~ được trình bày cụ thể trong Chương 5.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát hiện trạng từ ba nguồn chính bao gồm người dùng, hệ thống hiện có, và ứng dụng tương tự, qua đó xác định yêu cầu chức năng thông qua mười trường hợp sử dụng, đặc tả chi tiết năm trường hợp sử dụng quan trọng nhất, và đưa ra các yêu cầu phi chức năng về hiệu năng, bảo mật, cũng như hạ tầng hệ thống.

Chương 3 giới thiệu các công nghệ và nền tảng kiến trúc được lựa chọn để hiện thực hóa các yêu cầu đã xác định ở Chương 2, bao gồm Turborepo, NestJS, Next.js, Prisma, Mongoose, Redis, MinIO, và Docker, cùng lý do chọn mỗi công nghệ thay vì các giải pháp thay thế có sẵn trên thị trường.

Chương 4 trình bày thiết kế kiến trúc tổng quan theo mô hình phân tầng, thiết kế giao diện cho trang quản trị và trang hiển thị cửa hàng, thiết kế lớp cho các thành phần chính, lược đồ cơ sở dữ liệu PostgreSQL và MongoDB, cùng với kết quả xây dựng ứng dụng, quy trình kiểm thử và mô hình triển khai hệ thống.

Chương 5 phân tích chuyên sâu ba đóng góp kỹ thuật nổi bật của đề tài, bao gồm cơ chế cô lập dữ liệu đa thuê bao bằng AsyncLocalStorage, kiến trúc giao diện “Zero-file” với động cơ kết xuất thành phần động, và mô hình Master Template với quy trình cá nhân hóa giao diện hai pha nháp–xuất bản.

Chương 6 tổng kết các kết quả đã đạt được trong quá trình nghiên cứu và phát triển, so sánh sản phẩm hoàn thiện với các nền tảng tương tự trên thị trường, đánh giá một cách khách quan những điểm chưa hoàn thiện và đề xuất các hướng phát triển tiếp theo.

~~Chương này đã làm rõ bài toán thực tiễn về sự thiếu hụt một nền tảng phù hợp cho các doanh nghiệp vừa và nhỏ, đồng thời phân tích những điểm còn hạn chế của các hệ thống thương mại điện tử hiện tại. Dựa trên cơ sở đó, đề tài đề xuất việc phát triển hệ thống ShopVolo v2 ứng dụng kiến trúc đa thuê bao và nguyên lý giao diện~~

~~"Zero-file" để tối ưu hóa quá trình vận hành và tùy biến. Chương tiếp theo sẽ trình bày chi tiết kết quả khảo sát hiện trạng và phân tích yêu cầu của hệ thống.~~

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán về rào cản chi phí và kỹ thuật mà các doanh nghiệp vừa và nhỏ gặp phải khi tiếp cận thương mại điện tử, đồng thời đề xuất giải pháp nền tảng ShopVolo v2. Chương 2 này đi sâu vào việc khảo sát và phân tích yêu cầu của hệ thống để làm cơ sở cho quá trình thiết kế. Cụ thể, mục 2.1 trình bày kết quả khảo sát hiện trạng từ ba nguồn chính nhằm nhận diện rõ vấn đề. Tiếp đó, mục 2.2 cung cấp cái nhìn tổng quan về các chức năng của hệ thống thông qua các biểu đồ ca sử dụng và biểu đồ hoạt động. Mục 2.3 đặc tả chi tiết luồng nghiệp vụ của các trường hợp sử dụng cốt lõi. Cuối cùng, mục 2.4 đưa ra các yêu cầu phi chức năng mà hệ thống cần đáp ứng.

2.1 Khảo sát hiện trạng

Đối tượng người dùng của ShopVolo v2 là các doanh nghiệp vừa và nhỏ tại Việt Nam có nhu cầu mở rộng kênh bán hàng trực tuyến. Theo báo cáo về thương mại điện tử Việt Nam [15], chi phí xây dựng và duy trì website là một trong những rào cản lớn nhất đối với nhóm doanh nghiệp này. Người dùng cần một nền tảng dễ sử dụng, không yêu cầu kiến thức lập trình, có chi phí duy trì thấp nhưng vẫn cung cấp được giao diện chuyên nghiệp. Từ khảo sát đó, hệ thống xác định hai nhóm tác nhân:

- Quản trị nền tảng (Platform Admin): người vận hành hệ thống, quản lý master template (bộ giao diện mặc định) dùng ~~chính: (i) chủ chung~~ cho toàn bộ cửa hàng ~~(Tenant Owner) là~~.
- Chủ cửa hàng (Tenant Owner): người kinh doanh trực tiếp thiết lập và quản lý cửa hàng của mình, ~~và (ii) khách hàng (End Customer) là~~.
- Khách hàng (End Customer): người truy cập trang cửa hàng để xem và mua sản phẩm.

Trong đó, chủ cửa hàng và khách hàng là hai nhóm người dùng trong tâm mà đề tài hướng tới.

Xét các hệ thống mà nhóm người dùng này đang sử dụng, phổ biến nhất là hai hình thức: tự dựng website bằng WordPress kết hợp WooCommerce, hoặc bán hàng hoàn toàn trên mạng xã hội như Facebook và Zalo. Cả hai hình thức đều bộc lộ hạn chế rõ rệt: không có hệ thống quản lý đơn hàng và tồn kho tập trung, trải nghiệm giao diện không nhất quán giữa các thiết bị, và khó mở rộng khi số lượng sản phẩm cũng như lượng truy cập tăng lên.

Xét các ứng dụng tương tự trên thị trường, ~~Shopify~~:

- [Shopify](#) là nền tảng SaaS hàng đầu với giao diện chuyên nghiệp, nhưng chi phí duy trì hàng tháng cao và khả năng tùy biến sâu bị giới hạn trong khuôn khổ các theme dựng sẵn [3]. ~~WooCommerce~~
- [WooCommerce](#) mã nguồn mở cho tự do tùy biến tối đa, nhưng buộc người dùng tự thuê máy chủ, tự cấu hình bảo mật và cập nhật thường xuyên, đòi hỏi nguồn lực kỹ thuật đáng kể [10]. ~~Ở thị trường trong nước, Haravan~~
- [Haravan](#) tích hợp tốt các dịch vụ vận chuyển và thanh toán nội địa, song công cụ tùy biến giao diện còn hạn chế đối với người không biết lập trình [11]. ~~Với phân khúc doanh nghiệp lớn, Adobe Commerce (Magento) rất mạnh nhưng phức tạp và tốn kém để triển khai lần duy trì [12].~~

Tiêu chí so sánh	Shopify	WooCommerce WooCommerce	Haravan	ShopVolo v2 (đề xuất)
Mô hình triển khai	SaaS	Tự vận hành	SaaS	SaaS
Chi phí hàng tháng	\$29-\$299 Từ 39 USD	Miễn phí (phần lãi , tự thuê hosting)	200 nghìn-3 300 nghìn - 1,5 triệu VND đồng	Tối ưu nhờ dùng Chia sẻ chung hạ tầng
Tùy biến giao diện	Theme dựng sẵn	Cần lập trình viên	Hạn chế	Kéo thả từng khối, không cần lập trình
Đa thuê bao thực sự	Có	Không	Không	Có (AsyncLocalStorage Async-LocalStorage)
Thanh toán phù hợp nội địa Việt Nam	Hạn chế	Qua plugin	Có	Có (COD, chuyển khoản QR, ví)
Vận hành nhiều Chi phí biên cho mỗi cửa hàng trên hạ tầng nhỏ	Không áp dụng	Không Cao (mỗi cửa hàng một hạ tầng)	Không áp dụng	Mục tiêu thiết kế Thấp (vài tài liệu JSON)

Bảng 2.1: So sánh chi tiết các nền tảng thương mại điện tử

Bảng 2.1 cho thấy khoảng trống của thị trường: chưa nền tảng nào cân bằng được đồng thời chi phí ~~, khả năng vận hành nhiều~~ [biên thấp cho mỗi](#) cửa hàng ~~trên hạ tầng giới hạn~~ và công cụ tùy biến giao diện dành cho người không chuyên. Từ đó, đề tài xác định 10 chức năng cốt lõi cần xây dựng: (i) ~~khởi~~

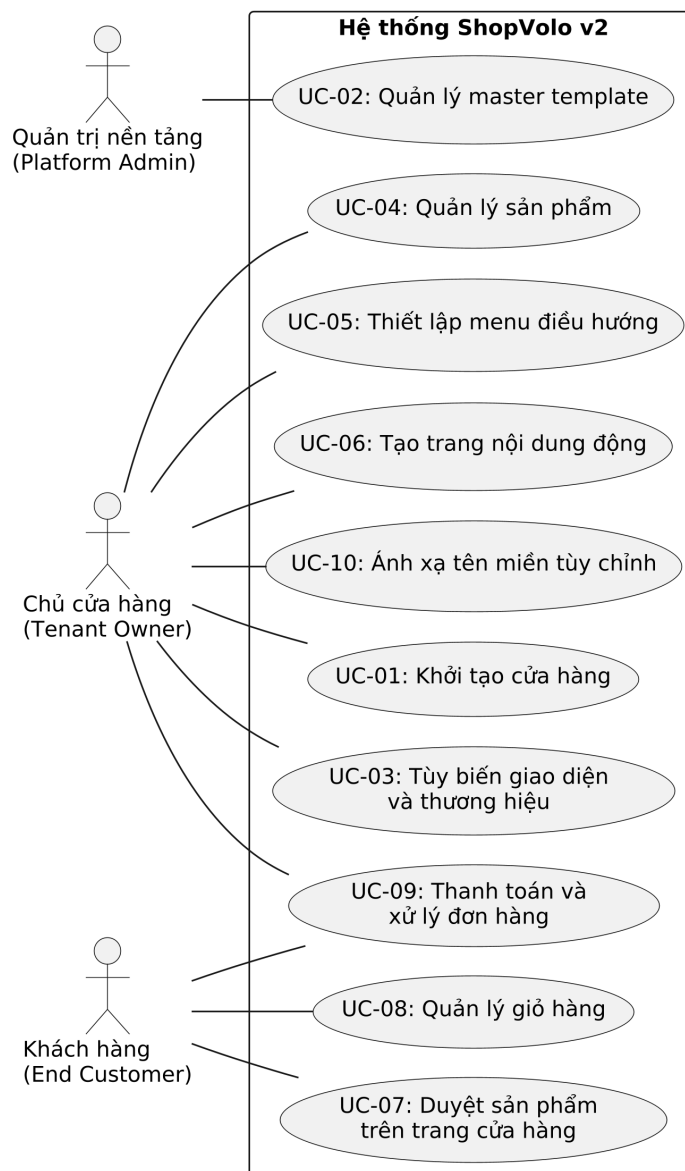
1. [Khởi](#) tạo cửa hàng, ~~(ii) quản lý chương trình khuyến mãi, (iii) tùy~~

2. Tùy biến giao diện và thương hiệu, ~~(iv) quản.~~
3. Quản lý sản phẩm, ~~(v) thiết lập menu điều hướng, (vi) tạo.~~
4. Tạo trang nội dung động, ~~(vii) duyệt.~~
5. Duyệt sản phẩm trên trang cửa hàng, ~~(viii) quản.~~
6. Quản lý giỏ hàng, ~~(ix) thanh.~~
7. Thanh toán và xử lý đơn hàng, ~~và (x) ánh xạ tên miền tùy chỉnh.~~

2.2 Tổng quan chức năng

Phần này mô tả cấu trúc chức năng của hệ thống thông qua biểu đồ ca sử dụng từ mức tổng quát đến mức phân rã, cùng các biểu đồ hoạt động cho những quy trình nghiệp vụ quan trọng. Việc đặc tả chi tiết từng chức năng cốt lõi được trình bày ở mục 2.3.

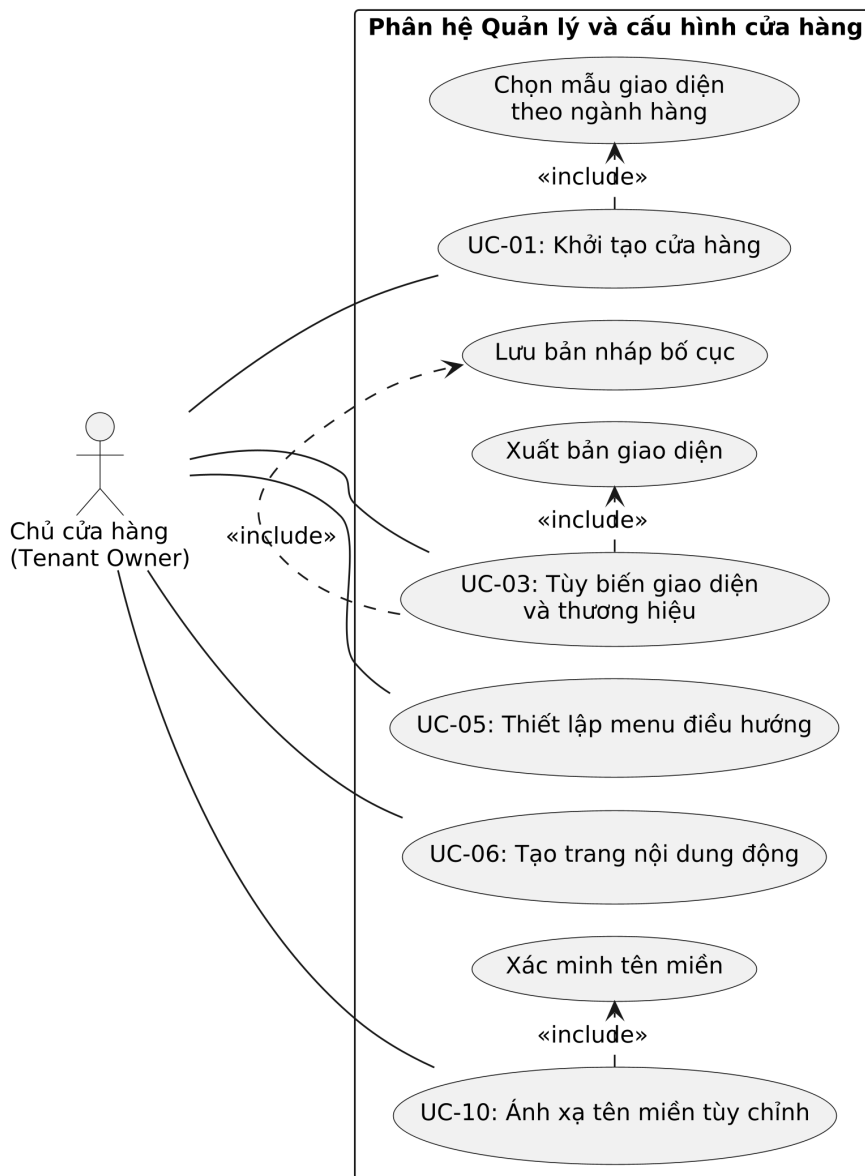
2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát hệ thống ShopVolo v2

Hình 2.1 thể hiện toàn cảnh tương tác giữa **hai** **ba** tác nhân và hệ thống. Tác nhân quản trị nền tảng phụ trách quản lý master template — bộ giao diện mặc định mà mọi cửa hàng kế thừa. Tác nhân chủ cửa hàng thực hiện nhóm chức năng quản trị cửa hàng: khởi tạo cửa hàng, quản lý sản phẩm và **khuyến mãi**, tùy biến giao diện và thương hiệu, thiết lập menu điều hướng, tạo trang nội dung động và ảnh xạ tên miền; tác nhân này cũng tham gia ca sử dụng thanh toán và xử lý đơn hàng ở vai trò xử lý các đơn phát sinh. Tác nhân khách hàng tương tác với mặt tiền cửa hàng: duyệt sản phẩm, quản lý giỏ hàng và thực hiện thanh toán.

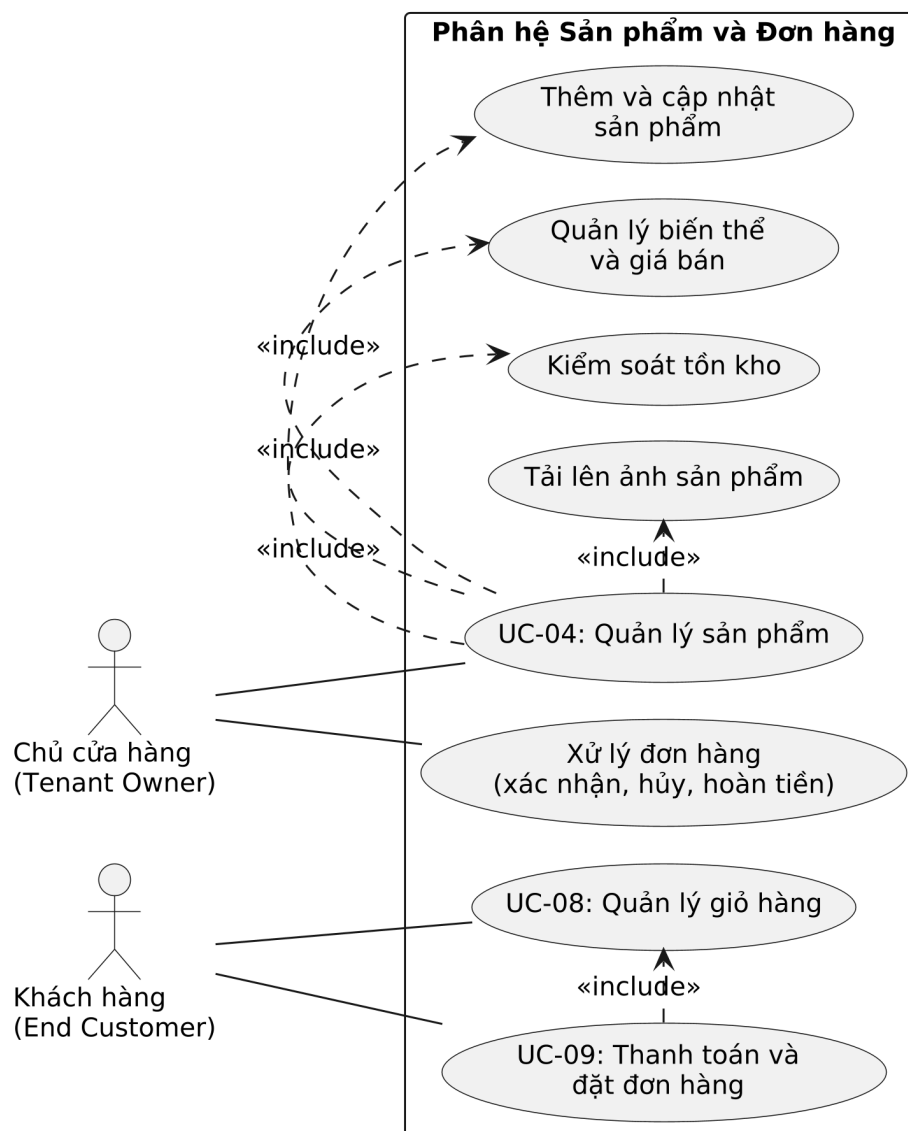
2.2.2 Biểu đồ use case phân rã — Quản lý cửa hàng



Hình 2.2: Biểu đồ use case phân rã nhóm Quản lý cửa hàng

Hình 2.2 phân rã nhóm chức năng thiết lập cửa hàng. Ca sử dụng khởi tạo cửa hàng bao gồm bước chọn mẫu giao diện theo ngành hàng, nhờ đó cửa hàng mới có ngay giao diện hoàn chỉnh. Ca sử dụng tùy biến giao diện bao gồm hai bước con bắt buộc là lưu bản nháp và xuất bản, phản ánh quy trình chỉnh sửa hai pha của hệ thống. Việc ánh xạ tên miền tùy chỉnh bao gồm bước xác minh quyền sở hữu tên miền trước khi kích hoạt.

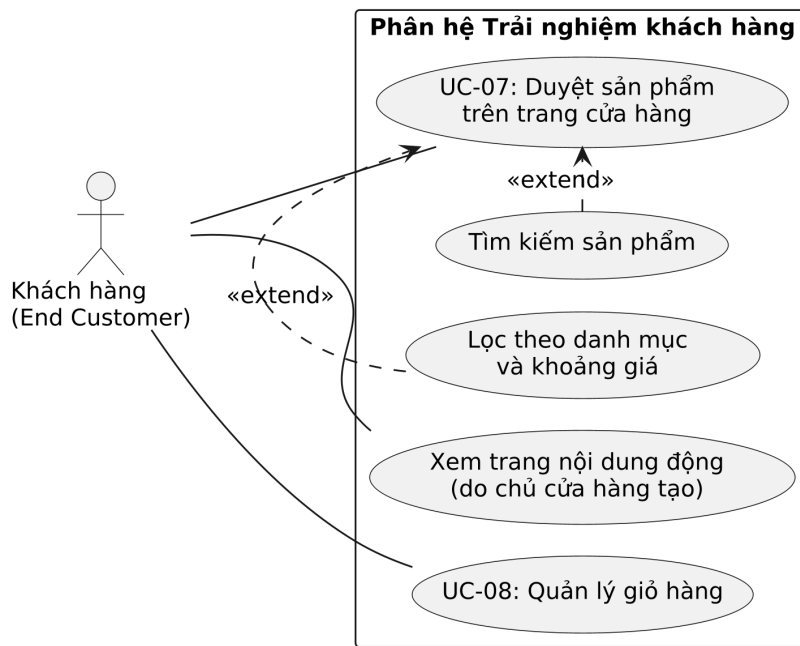
2.2.3 Biểu đồ use case phân rã — Sản phẩm và Đơn hàng



Hình 2.3: Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng

Hình 2.3 mô tả các chức năng nghiệp vụ bán hàng. Quản lý sản phẩm bao gồm bốn tác vụ con: thêm và cập nhật sản phẩm, quản lý biến thể và giá bán, kiểm soát tồn kho, và tải lên ảnh sản phẩm. Về phía khách hàng, ca sử dụng thanh toán bao gồm việc đọc giỏ hàng (khi khách đặt từ giỏ); sau khi đơn được tạo, chủ cửa hàng tiếp nhận và xử lý đơn qua các thao tác xác nhận, hủy hoặc hoàn tiền.

2.2.4 Biểu đồ use case phân rã — Trải nghiệm khách hàng



Hình 2.4: Biểu đồ use case phân rã nhóm Trải nghiệm khách hàng

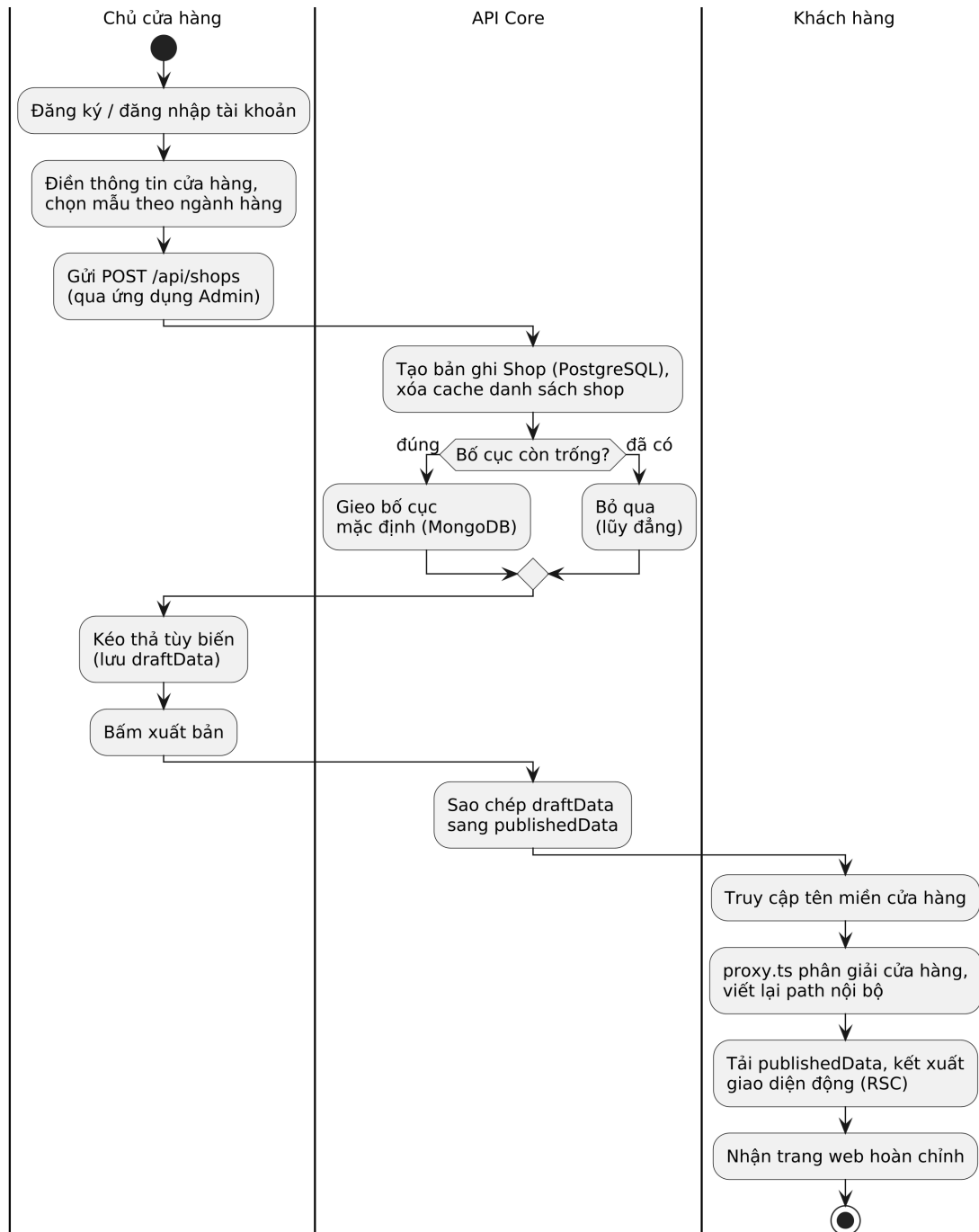
Hình 2.4 mô tả trải nghiệm phía người mua. Khách hàng xem các trang nội dung do chủ cửa hàng tạo và duyệt danh mục sản phẩm được kết xuất động từ cấu hình. Hai ca sử dụng mở rộng là lọc theo danh mục cùng khoảng giá và tìm kiếm sản phẩm, chỉ kích hoạt khi khách hàng có nhu cầu thu hẹp kết quả.

2.2.5 Quy trình nghiệp vụ — Khởi tạo cửa hàng và kết xuất giao diện

Hình 2.5 mô tả quy trình nghiệp vụ xuyên suốt nhất của hệ thống, nổi từ lúc cửa hàng được khởi tạo đến lúc khách hàng nhìn thấy trang web. Sau khi chủ cửa hàng đăng ký và điền thông tin kèm mẫu ngành hàng, ứng dụng Admin lần lượt gọi hai API: tạo bản ghi cửa hàng và gieo bố cục khởi điểm; bước gieo chỉ ghi vào các tài liệu còn trống nên có thể gọi lại an toàn. Chủ cửa hàng sau đó tùy biến giao diện trên bản nháp và bấm xuất bản. Khi khách truy cập tên miền của cửa hàng, ứng dụng Storefront bóc tách định danh, tải bản bố cục đã xuất bản cùng dữ liệu sản phẩm rồi kết xuất giao diện hoàn chỉnh.

2.3 Đặc tả chức năng

Trong 10 chức năng đã xác định ở mục 2.1, phần này đặc tả chi tiết năm trường hợp sử dụng có ảnh hưởng lớn nhất đến thiết kế hệ thống.



Hình 2.5: Biểu đồ hoạt động quy trình khởi tạo cửa hàng và kết xuất giao diện động

2.3.1 Đặc tả use case: Khởi tạo cửa hàng (UC-01)

Tên use case	Khởi tạo cửa hàng
Tác nhân	Chủ cửa hàng
Mục đích	Tạo cửa hàng mới với giao diện khởi điểm hoạt động được ngay
Tiền điều kiện	Chủ cửa hàng đã đăng nhập tài khoản; tên miền dự kiến chưa thuộc cửa hàng nào
Hậu điều kiện	Bản ghi cửa hàng được tạo trong PostgreSQL; bố cục khởi điểm (đầu trang, chân trang và ba trang mặc định) được gieo vào MongoDB; cache danh sách cửa hàng của chủ sở hữu được làm mới
Luồng chính	1. Chủ cửa hàng điền tên cửa hàng, tên miền, đơn vị tiền tệ và chọn mẫu giao diện theo ngành hàng 2. Ứng dụng Admin gửi POST /api/shops với name, domain, currency, templateKey 3. Hệ thống tạo bản ghi Shop gắn với tài khoản chủ sở hữu 4. Hệ thống xóa cache danh sách cửa hàng của người dùng để quyền sở hữu có hiệu lực ngay 5. Ứng dụng Admin gọi POST /api/layouts/{shopId}/seed 6. Hệ thống gieo bố cục mặc định vào các tài liệu còn trống (thao tác lũy đẳng) 7. Ứng dụng Admin chuyển sang trình hướng dẫn tám bước hoàn thiện cửa hàng
Luồng phát sinh	[2a] Tên miền đã tồn tại (vi phạm ràng buộc duy nhất) → trả lỗi, yêu cầu chọn tên khác [2b] Người dùng chưa đăng nhập → HTTP 401 [5a] Bước gieo bố cục thất bại → không chặn luồng; trình thiết kế tự dùng bố cục mặc định phía máy khách

Bảng 2.2: Đặc tả use case Khởi tạo cửa hàng (UC-01)

Bảng 2.2 mô tả luồng khởi tạo: điểm đáng chú ý là tính lũy đẳng của bước gieo bố cục và việc chủ động làm mới cache quyền sở hữu, hai chi tiết bảo đảm trải nghiệm nhất quán ngay sau khi tạo cửa hàng.

2.3.2 Đặc tả use case: Tùy biến giao diện và thương hiệu (UC-03)

Tên use case	Tùy biến giao diện và thương hiệu
Tác nhân	Chủ cửa hàng
Mục đích	Thay đổi bố cục, thành phần, màu sắc và phông chữ của trang cửa hàng bằng thao tác kéo thả
Tiền điều kiện	Chủ cửa hàng đã đăng nhập; cửa hàng đang hoạt động
Hậu điều kiện	Bản nháp được lưu vào trường draftData; sau khi xuất bản, publishedData được cập nhật và khách truy cập thấy giao diện mới trong tối đa 60 giây
Luồng chính	1. Chủ cửa hàng mở trình thiết kế; hệ thống tải bản nháp hiện có (nếu nháp trống thì sao lưu từ bản đã xuất bản) 2. Chủ cửa hàng kéo thả thành phần, chỉnh thuộc tính qua biểu mẫu sinh tự động từ lược đồ thành phần 3. Trình thiết kế gửi POST /api/layouts/builder/save/global (hoặc save/page) để lưu draftData 4. Chủ cửa hàng xem trước ở chế độ máy tính hoặc di động 5. Chủ cửa hàng bấm xuất bản; hệ thống sao chép draftData sang publishedData (từng trang hoặc toàn bộ) 6. Bộ nhớ đệm bố cục phía Storefront hết hạn theo chu kỳ 60 giây và tải bản mới
Luồng phát sinh	[5a] Chưa có bản nháp nào → hệ thống báo lỗi “No layout draft found” [5b] Trang chưa từng chỉnh sửa → thao tác xuất bản bỏ qua trang đó

Bảng 2.3: Đặc tả use case Tùy biến giao diện và thương hiệu (UC-03)

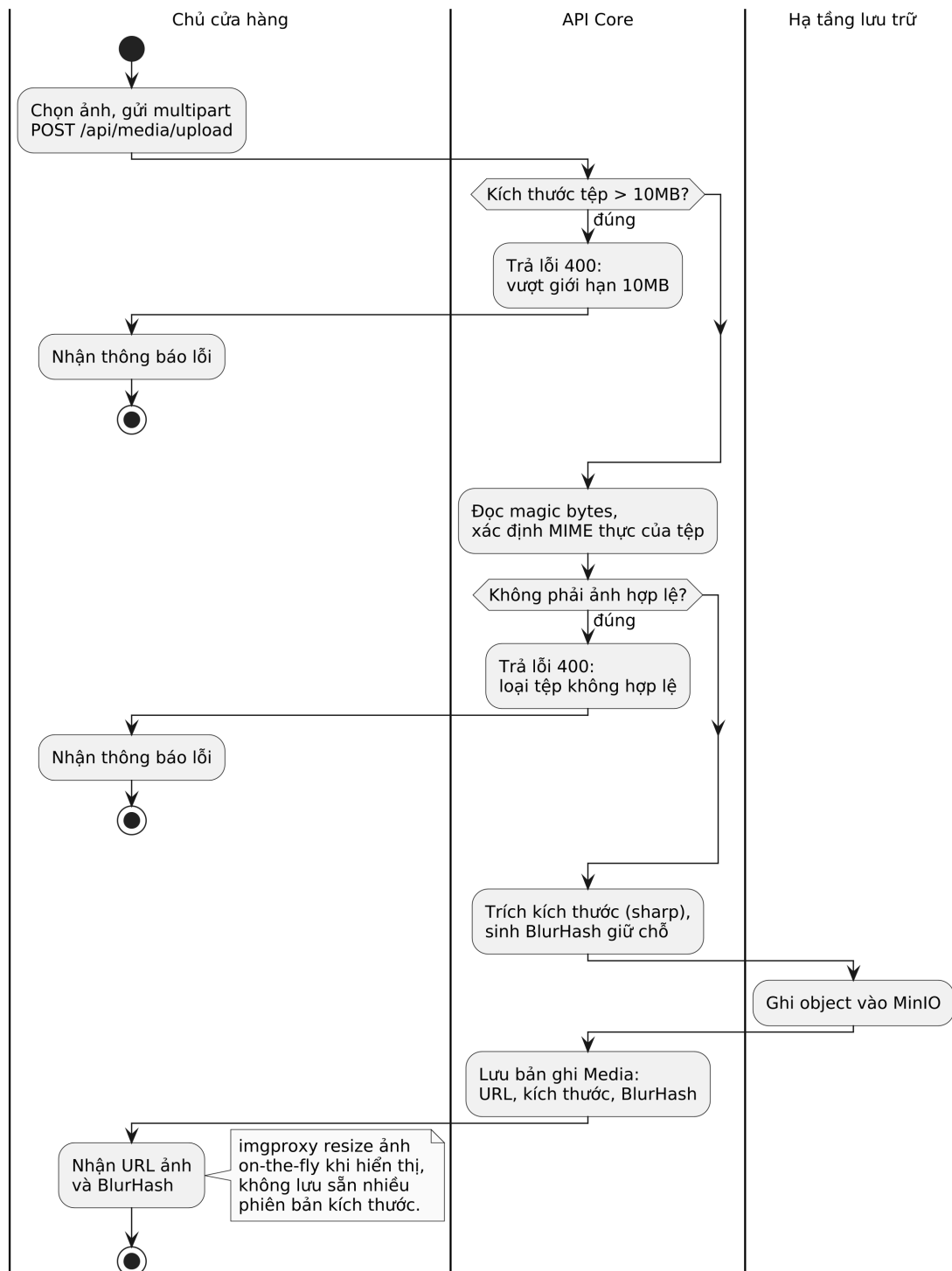
Bảng 2.3 thể hiện quy trình chỉnh sửa hai pha: mọi thao tác chỉ chạm vào bản nháp, còn khách truy cập luôn nhìn thấy bản đã xuất bản; cơ chế này được phân tích sâu tại mục 5.3.

2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04)

Tên use case	Quản lý sản phẩm
Tác nhân	Chủ cửa hàng
Mục đích	Thêm, chỉnh sửa, xóa sản phẩm cùng biến thể, giá bán, tồn kho và hình ảnh
Tiền điều kiện	Chủ cửa hàng đã đăng nhập vào phiên quản trị của đúng cửa hàng
Hậu điều kiện	Thông tin sản phẩm và biến thể được lưu trong PostgreSQL; ảnh gốc nằm trên MinIO kèm bản ghi Media chứa URL và mã BlurHash
Luồng chính	1. Chủ cửa hàng điền tên, mô tả, danh mục và các biến thể (SKU, giá, tồn kho) của sản phẩm 2. Chủ cửa hàng tải ảnh lên; hệ thống kiểm tra dung lượng (tối đa 10MB) và loại tệp thực qua magic bytes 3. Hệ thống trích kích thước ảnh và sinh mã BlurHash làm ảnh giữ chỗ 4. Ảnh gốc được ghi vào MinIO; bản ghi Media được lưu kèm URL và metadata 5. Hệ thống lưu sản phẩm; ràng buộc SKU duy nhất trong phạm vi cửa hàng 6. Khi hiển thị, ảnh được imgproxy biến đổi đúng kích thước và chất lượng theo thiết bị
Luồng phát sinh	[2a] Tệp vượt 10MB hoặc không phải ảnh hợp lệ → trả lỗi 400 kèm lý do [5a] SKU trùng trong cùng cửa hàng → từ chối và yêu cầu đổi mã

Bảng 2.4: Đặc tả use case Quản lý sản phẩm (UC-04)

Bảng 2.4 và hình 2.6 mô tả quy trình quản lý sản phẩm, trong đó luồng xử lý ảnh xác thực loại tệp bằng magic bytes thay vì tin vào phần mở rộng, đồng thời sinh BlurHash để trang web hiển thị ảnh giữ chỗ mượt mà trong lúc tải.



Hình 2.6: Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm

2.3.4 Đặc tả use case: Duyệt sản phẩm trên trang cửa hàng (UC-07)

Tên use case	Duyệt sản phẩm trên trang cửa hàng
Tác nhân	Khách hàng
Mục đích	Hiển thị trang cửa hàng với giao diện riêng của từng cửa hàng và dữ liệu sản phẩm thực
Tiền điều kiện	Tên miền cửa hàng trở đúng hệ thống; cửa hàng không bị đình chỉ
Hậu điều kiện	Trang được kết xuất hoàn chỉnh phía máy chủ; bố cục và dữ liệu được lưu vào bộ nhớ đệm cho các lượt truy cập sau
Luồng chính	1. Khách hàng truy cập tên miền của cửa hàng 2. Tầng middleware của Storefront bóc tách định danh cửa hàng từ tên miền và viết lại đường dẫn nội bộ 3. Trang máy chủ tải song song bố cục đã xuất bản (GET /api/layouts/{shopId}/page/...) và thông tin cửa hàng, kèm header x-shop-id 4. Kết quả được lưu vào Next.js Data Cache với chu kỳ làm mới 60 giây (bố cục) và 30 giây (sản phẩm) 5. Động cơ kết xuất duyệt danh sách thành phần, ánh xạ componentId sang React component và bơm dữ liệu sản phẩm thực 6. HTML hoàn chỉnh được trả về trình duyệt
Luồng phát sinh	[2a] Cửa hàng bị đình chỉ → HTTP 403, hiển thị thông báo tạm dừng dịch vụ [3a] Cửa hàng chưa cấu hình bố cục → hiển thị trang giới thiệu mặc định kèm liên kết xem sản phẩm [5a] Thành phần không tồn tại trong bảng ánh xạ → hiển thị khung cảnh báo tại đúng vị trí, phần còn lại của trang vẫn hoạt động

Bảng 2.5: Đặc tả use case Duyệt sản phẩm trên trang cửa hàng (UC-07)

Bảng 2.5 mô tả luồng đọc quan trọng nhất của hệ thống — nơi kiến trúc Zero-file phát huy tác dụng: toàn bộ trang được dựng từ dữ liệu JSON và bảng ánh xạ thành phần, không có tệp giao diện riêng cho bất kỳ cửa hàng nào. Chi tiết cơ chế được trình bày tại mục 5.2.

2.3.5 Đặc tả use case: Thanh toán và đặt đơn hàng (UC-09)

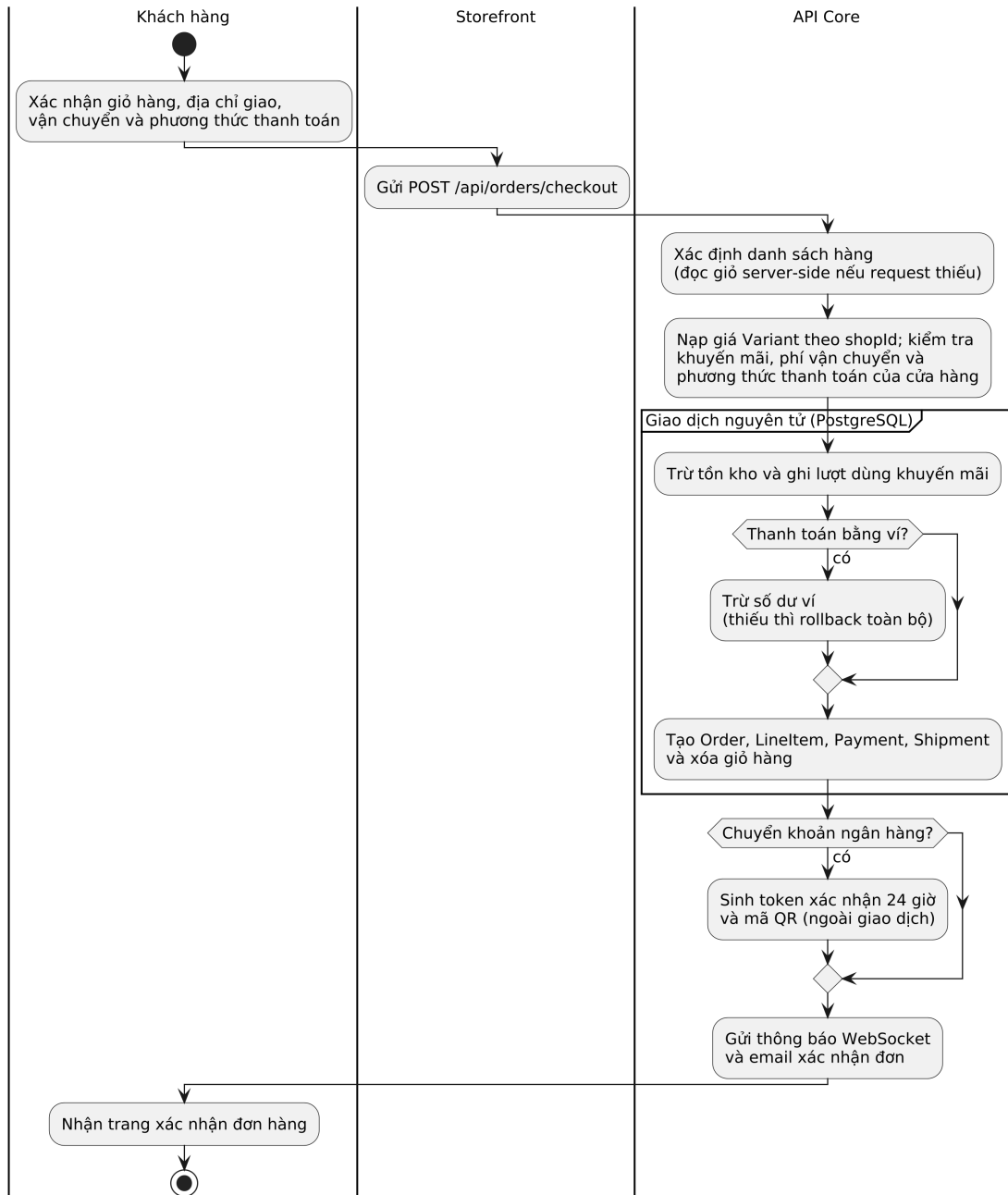
Tên use case	Thanh toán và đặt đơn hàng
Tác nhân	Khách hàng
Mục đích	Chuyển giỏ hàng thành đơn hàng có trạng thái rõ ràng và ghi nhận thanh toán
Tiền điều kiện	Khách hàng đã đăng nhập tài khoản mua hàng và có ít nhất một sản phẩm muốn đặt
Hậu điều kiện	Đơn hàng, các dòng hàng, bản ghi thanh toán và vận đơn được tạo nhất quán; tồn kho bị trừ; giỏ hàng được dọn; khách nhận thông báo và email xác nhận
Luồng chính	1. Khách xác nhận giỏ hàng, địa chỉ giao, phương thức vận chuyển và phương thức thanh toán 2. Storefront gửi POST /api/orders/checkout; nếu thiếu danh sách hàng, hệ thống đọc giỏ phía máy chủ 3. Hệ thống nạp giá các biến thể thuộc đúng cửa hàng và kiểm tra mã khuyến mãi (nếu có) 4. Hệ thống tính phí vận chuyển và xác thực phương thức thanh toán thuộc cửa hàng 5. Trong một giao dịch nguyên tử: trừ tồn kho, ghi lượt dùng khuyến mãi, trừ ví (nếu thanh toán ví), tạo đơn kèm dòng hàng, tạo bản ghi thanh toán, dọn giỏ, tạo vận đơn 6. Với chuyển khoản ngân hàng: hệ thống phát hành token xác nhận hiệu lực 24 giờ và sinh mã QR sau khi giao dịch hoàn tất 7. Hệ thống gửi thông báo thời gian thực và email xác nhận đơn
Luồng phát sinh	[2a] Giỏ hàng trống và không có danh sách hàng → lỗi 400 [3a] Biến thể không thuộc cửa hàng, khuyến mãi hết hạn hoặc vượt giới hạn lượt dùng → lỗi 400 [5a] Thiếu tồn kho hoặc số dư ví không đủ → hoàn tác toàn bộ giao dịch, không phát sinh đơn [6a] Token xác nhận hết hạn hoặc đã dùng → từ chối nâng trạng thái thanh toán

Bảng 2.6: Đặc tả use case Thanh toán và đặt đơn hàng (UC-09)

Bảng 2.6 và hình 2.7 cho thấy tính nhất quán dữ liệu là trọng tâm của luồng này: mọi thao tác ghi đều nằm trong một giao dịch nguyên tử nên không thể xảy ra trạng thái nửa vời như trừ kho mà không có đơn.

2.4 Yêu cầu phi chức năng

Về hiệu năng, mục tiêu thiết kế của hệ thống là ~~phục vụ đồng thời 20.000 giữ~~ chi phí tài nguyên tăng thêm cho mỗi cửa hàng (chi phí biên) ở mức thấp, để nhiều



Hình 2.7: Biểu đồ hoạt động quy trình thanh toán và đặt đơn hàng

cửa hàng có thể cùng chạy trên một ~~máy chủ 16GB RAM, với thời gian hạ tầng~~ dùng chung mà vẫn duy trì trải nghiệm phản hồi ~~dưới 500ms cho~~ nhanh. Do toàn bộ cửa hàng chia sẻ một ứng dụng duy nhất thay vì mỗi cửa hàng một tiến trình riêng, bộ nhớ tiêu tốn không tăng tuyến tính theo số cửa hàng. Về độ trễ, các yêu cầu ~~chưa~~ đã có dữ liệu trong bộ nhớ đệm ~~và dưới 10ms cho dữ liệu đã~~ được phục vụ gần như tức thời do đọc trực tiếp từ RAM, còn các yêu cầu chưa được đệm chỉ phụ thuộc vào một vài truy vấn cơ sở dữ liệu. Để ~~hướng tới mục tiêu đạt được điều~~ này, mọi thành phần đều ~~phải~~ khai báo giới hạn tài nguyên tường minh khi triển khai (ví dụ dịch vụ API giới hạn 350MB và Redis giới hạn 512MB bộ nhớ trong cấu hình Kubernetes), đồng thời các truy vấn có tần suất cao ~~phải~~ đều đi qua bộ nhớ đệm: định danh cửa hàng được đệm trên Redis với thời gian sống 300 giây, còn bố cục giao diện và dữ liệu sản phẩm được đệm ở tầng kết xuất với chu kỳ làm mới lần lượt 60 và 30 giây.

Về độ tin cậy, hệ thống đặt mục tiêu duy trì thời gian hoạt động từ 99,5% trở lên. Cơ sở dữ liệu phải được sao lưu tự động hằng ngày (hiện thực bằng CronJob chạy lúc ~~3-2~~ giờ sáng). Mọi lỗi phát sinh từ tầng dữ liệu phải được chuyển thành ngoại lệ có kiểm soát của NestJS, tuyệt đối không để lộ chi tiết truy vết nội bộ trong phản hồi trả về máy khách. Lỗi kết xuất của một thành phần giao diện không được làm sập toàn trang.

Về tính dễ sử dụng, toàn bộ bảng điều khiển Admin phải dùng được bởi người không có kiến thức kỹ thuật: các thao tác trọng yếu như tải ảnh sản phẩm hay đổi màu thương hiệu phải hoàn thành trong tối đa 3 phút, và cửa hàng mới khởi tạo phải có giao diện hoạt động được ngay thay vì trang trắng. Giao diện phải đáp ứng trên cả thiết bị di động, máy tính bảng và máy tính để bàn.

Về bảo mật, hệ thống xác thực bằng phiên đăng nhập do thư viện Better Auth quản lý, tách biệt hoàn toàn phiên của chủ cửa hàng và phiên của khách mua hàng. Yêu cầu cốt lõi của môi trường đa thuê bao là cô lập dữ liệu tuyệt đối: chủ cửa hàng này không thể truy cập dữ liệu của cửa hàng khác trên cùng hạ tầng. Các khóa bí mật chỉ được nạp qua biến môi trường và không xuất hiện trong mã nguồn; toàn bộ kết nối ra Internet phải được mã hóa TLS.

Về yêu cầu kỹ thuật, hệ thống chạy trên Node.js phiên bản 22 trở lên; phía máy khách hỗ trợ các trình duyệt Chrome và Firefox từ phiên bản 100, Safari từ phiên bản 15. Tầng dữ liệu sử dụng PostgreSQL 15 với Prisma ORM và MongoDB 6 với Mongoose. Môi trường phát triển vận hành bằng Docker Compose, môi trường sản xuất hướng tới Kubernetes; toàn bộ mã nguồn được quản lý theo mô hình Monorepo bằng Turborepo để tối ưu thời gian xây dựng.

Chương này đã khảo sát hiện trạng từ ba nguồn, xác định 10 nhóm chức năng cốt lõi, đặc tả chi tiết năm trường hợp sử dụng trung tâm và đưa ra các yêu cầu phi chức năng của hệ thống. Chương tiếp theo trình bày các công nghệ được lựa chọn để hiện thực hóa các yêu cầu này.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chức năng và phi chức năng của hệ thống ~~ShopVolo v2 từ quá trình khảo sát kỹ lưỡng thực tiễn người dùng~~. Chương 3 ~~này sẽ~~ trình bày các công nghệ được lựa chọn để đáp ứng những yêu cầu đó, ~~đồng thời~~. Mỗi công nghệ được đặt trong cùng một khung phân tích rõ lý do tại sao chúng được tin dùng hơn so với các ~~gồm ba phần~~:

- Yêu cầu cụ thể mà công nghệ phải giải quyết.
- Các giải pháp thay thế trên thị trường. Về mặt tổng thể, nội dung sẽ được liệt kê sơ bộ thông qua các ~~đã~~ cân nhắc.
- Lý do lựa chọn.

Các nhóm công nghệ ~~lỗi như kiến trúc~~ được trình bày theo trình tự của một request đi qua hệ thống: công cụ quản lý ~~tập trung~~ mã nguồn, ~~hệ thống lập trình máy chủ phụ trợ, nền tảng phát triển giao diện hiển thị, cấu trúc framework backend, framework frontend, hai~~ cơ sở dữ liệu ~~chuyên biệt~~, ~~bộ nhớ đệm, lưu trữ ảnh, xác thực~~, và cuối cùng là ~~các công nghệ đóng gói~~ hạ tầng ~~cũng như lưu trữ đối tượng~~ triển khai.

3.1 Turborepo — Quản lý Monorepo

Để đáp ứng yêu cầu phi chức năng tại mục 2.4 về tính bảo trì và quy trình xây dựng tự động hóa, hệ ~~Hệ~~ thống yêu cầu quản lý ~~gồm~~ 4 ứng dụng (admin, api-core, storefront, cli-tool) và 5 gói ~~phần mềm~~ dùng chung (ui-registry, schema, database, i18n, master-templates) ~~một cách nhất quán~~ đặt trong cùng một kho ~~lưu trữ tập trung~~ mã nguồn. Yêu cầu đặt ra ở mục 2.4 là chia sẻ kiểu dữ liệu xuyên suốt giữa các ứng dụng và giữ thời gian build ngắn khi dự án lớn dần.

Các công cụ quản lý ~~kho chứa mã nguồn tập trung monorepo~~ phổ biến hiện nay ~~bao khác~~ được cân nhắc gồm (i) Nx, (ii) Lerna Nx, Lerna và (iii) ~~trình quản lý không gian làm việc~~ pnpm workspace ~~thuần túy của~~ pnpm.

Turborepo được chọn vì (i) ~~khả năng lưu trữ bộ nhớ đệm dạng tăng dần~~ giúp hệ thống ~~ba lý do~~:

- Bộ nhớ đệm theo tác vụ (task cache): chỉ ~~tiến hành biên dịch build~~ lại đúng các ~~những~~ gói ~~phần mềm~~ thực sự bị thay đổi, ~~giúp giảm triệt để nhờ đó rút ngắn đáng kể~~ thời gian build và thời gian chạy ~~các luồng tự động~~, (ii) công cụ này thiết lập CI.
- Lập lịch theo đồ thị phụ thuộc: build được ~~quy trình biên dịch tự động dựa~~

vào khái niệm sắp xếp theo đồ thị có hướng phi không chu trình (DAG) giữa các gói, và (iii) bảo đảm gói nền tảng được build trước gói phụ thuộc.

- **Tích hợp sẵn với npm workspace:** không yêu cầu quá trình cần chuyển đổi cấu trúc phức tạp khi tích hợp sẵn với không gian làm việc của npm đã tồn tại [16]- dự án đang có [16].

3.2 NestJS — Framework Backend

Để đáp ứng các chức năng được thiết kế chi tiết tại 2.3, hệ thống yêu cầu đã đặc tả ở mục 2.3 đòi hỏi một giải pháp xây dựng giao diện lập trình ứng dụng framework backend có cấu trúc mô-đun module rõ ràng như (quản lý cửa hàng, sản phẩm, đơn hàng, đồng thời), hỗ trợ tiêm phụ thuộc, xây dựng tầng trung gian phục vụ dependency injection và cho phép cài đặt middleware xử lý ngữ cảnh đa thuê bao và hỗ trợ định dạng tính cấu trúc mạnh mẽ cho mọi request.

Các công cụ lập trình phụ trợ bằng ngôn ngữ JavaScript nổi bật framework backend Node.js đáng cân nhắc khác bao gồm (i) Express.js, (ii) Fastify, Fastify và (iii) Hono.

NestJS được chọn vì (i) kiến ba lý do:

- **Tổ chức theo module:** cấu trúc mô-đun mô phỏng hệ thống Angular hỗ trợ tổ chức mã nguồn theo từng cụm lấy cảm hứng từ Angular tạo ranh giới rõ ràng, (ii) tích giữa các miền nghiệp vụ và giữ khả năng tách thành microservice về sau.
- **Hỗ trợ middleware toàn cục:** kết hợp rất tốt với bộ nhớ ngữ cảnh bất đồng bộ nhằm AsyncLocalStorage của Node.js để truyền tải định danh khách thuê một cách tenant (shopId) tự động, và (iii) sử dụng trình trang trí kết hợp với xuống mọi tầng xử lý.
- **Xử lý lỗi thống nhất:** cơ chế lọc ngoại lệ giúp decorator kết hợp exception filter cho phép xử lý thống nhất các lỗi nội bộ nhất quán trên toàn bộ máy chủ [17]- API [17].

Cụ thể, tầng trung gian tự động giải mã thông tin middleware đọc định danh khách thuê tenant từ phần mào đầu truy vấn, sau đó HTTP header rồi lưu vào bộ nhớ bất đồng bộ để toàn bộ quá trình đọc ghi AsyncLocalStorage, nhờ đó mọi truy vấn cơ sở dữ liệu tiếp theo phía sau đều tự động gắn kết đúng phạm vi cửa hàng; chi tiết cơ chế này được phân tích ở mục 5.1.

3.3 Next.js — Framework Frontend

Để đáp ứng yêu cầu hiệu năng tại 2.4, hệ thống ở mục 2.4 đòi hỏi nền tảng

mặt tiền phải trang cửa hàng phản hồi ở tốc độ dưới mức 500 mili-giây, cùng với đó trang quản trị nhanh, đồng thời bảng điều khiển của người bán phải tải xử lý được các dạng biểu mẫu cấu trúc phức tạp với độ tương tác phản ứng tức thì cao.

Các nền tảng xây dựng giao diện nổi bật thay thế bao framework frontend đáng cân nhắc khác gồm (i) bộ đôi React và Vite, (ii) Remix, React kết hợp Vite, Remix và (iii) hệ thống Nuxt.js dựa (trên Vu nền Vue).

Next.js phiên bản 16 được chọn vì (i) bộ định tuyến App Router với React Server Components ba lý do:

- App Router với React Server Components: cho phép kết xuất giao diện hoàn toàn render phía máy chủ (server-side rendering), giảm đáng kể dung khối lượng mã JavaScript tải về cho khách truy cập, (ii) trình duyệt.
- Middleware nhận diện tên miền: tầng middleware cho phép nhận diện tên miền của từng cửa hàng và viết lại (rewrite) đường dẫn ngay từ rất sớm trong đầu vòng đời request, và (iii) cơ chế Data Cache cho — nền tảng cho định tuyến đa thuê bao của trang cửa hàng.
- Next.js Data Cache: cho phép đặt chu kỳ làm mới và nhãn vô hiệu hóa (cache tag) riêng cho từng nguồn dữ liệu (— bổ cục, sản phẩm, thông tin cửa hàng) — một cách độc lập [18].

3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ

Để đáp ứng việc quản lý các Các thực thể có cấu trúc định hình cố định đã được xác định tại 2.1 như chủ ở mục 2.1 (cửa hàng, tài khoản, đơn vị mua hàng, hệ thống) đòi hỏi một giải pháp bảo đảm tính nguyên tử của giao dịch nguyên tử, ràng buộc chặt chẽ khóa ngoại chặt chẽ và khả năng trích xuất thông tin bảng truy vấn thống kê không cao.

Các giải pháp quản lý bảng biểu cấu trúc cơ sở dữ liệu quan hệ thay thế bao gồm (i) MySQL 8, (ii) SQLite ở SQLite (cho môi trường cục bộ,) và (iii) hệ thống CockroachDB (phân tán CockroachDB).

PostgreSQL 15 kết hợp công cụ Prisma được chọn vì (i) ba lý do:

- Giao dịch ACID: PostgreSQL bảo đảm giao dịch ACID tính nguyên tử cần thiết cho nghiệp vụ đặt hàng và trữ tồn kho, đồng thời hỗ trợ kiểu JSONB cho các trường tham số động, (ii) hệ ràng buộc.
- Ràng buộc toàn vẹn: khóa ngoại và ràng buộc duy nhất theo tổ hợp cột (ví dụ SKU duy nhất trong phạm vi một cửa hàng) phù hợp với thiết kế đa thuê bao ở mức hàng, và (iii) Prisma ORM (row-level).

- **Prisma ORM:** tự sinh kiểu dữ liệu TypeScript từ lược đồ và quản lý phiên bản các đợt di trú cấu trúc [19], [20]. migration [19], [20].

3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu

Để đáp ứng nền tảng kiến trúc Zero-file đã vạch ra tại 1.3, việc ở mục 1.3 lưu trữ các tệp cấu hình JSON phải giải quyết bài toán giao diện của hàng dưới dạng tài liệu JSON lồng nhau sâu sắc của bộ nhiều cấp (bố cục khung nền và bộ hợp nhất với phần tùy chỉnh biến của từng cửa hàng, đồng thời hỗ trợ cập nhật mở rộng), với cấu trúc thay đổi tùy theo cách dựng trang, nên không cần theo định dạng phù hợp với mô hình bảng cố định khắt khe của kho lưu trữ cơ sở dữ liệu quan hệ truyền thống.

Các nền tảng giải quyết cơ sở dữ liệu văn bản phi cấu trúc khác tài liệu thay thế gồm (i) CouchDB, (ii) giải pháp CouchDB, DynamoDB (đám mây DynamoDB,) và (iii) lợi việc tận dụng kiểu JSONB của chính cơ sở dữ liệu PostgreSQL.

MongoDB phiên bản 6 kết hợp thư viện Mongoose được chọn vì (i) mô hình tài liệu ba lý do:

- **Mô hình tài liệu:** biểu diễn tự nhiên các khối giao diện lồng nhau với độ sâu thay đổi theo từng cửa hàng, (ii).
- **Lưu hai phiên bản trong một tài liệu:** một tài liệu có thể chứa đồng thời hai phiên bản nháp (draftData) và bản xuất bản (publishedData) của bố cục, giúp nhờ đó thao tác xuất bản chỉ là một lệnh cập nhật nguyên tử, và (iii) Mongoose.
- **Mongoose:** cung cấp lớp kiểm soát lược đồ vừa đủ ở những các trường cốt lõi mà vẫn giữ tính linh hoạt cho phần cấu hình tự do [21].

Thiết kế chi tiết các collection được trình bày ở mục 4.2.3.

3.6 Redis — Cache In-memory — Bộ nhớ đệm trong RAM

Để đáp ứng yêu cầu hiệu năng cực kỳ khắt khe tại 2.4, hệ thống đặt mục tiêu phản hồi trong 10 mili-giây nhanh khi duyệt trang phục vụ cho việc lấy cấu hình ở phần 2.3.4, điều hoàn toàn bất khả thi mục 2.3.4 không thể đạt được nếu cứ liên tục gọi cấu trúc mỗi request đều phải truy vấn cơ sở dữ liệu thô để định danh cửa hàng.

Các phương pháp thay thế giải quyết pháp giảm độ trễ phổ biến bao thay thế gồm (i) Memcached, (ii) kiểm soát bộ nhớ qua giao thức HTTP cơ bản Memcached, đệm ở tầng HTTP và (iii) sử dụng mạng phân phối nội dung thuần túy việc chỉ dùng CDN.

Redis được chọn vì ~~(i) ba lý do:~~

- **Độ trễ thấp:** dữ liệu nằm hoàn toàn ~~trên bộ nhớ trong~~ trong RAM nên độ trễ đọc ở mức dưới một mili-giây, ~~(ii).~~
- **Tự hết hạn:** mỗi khóa được gán thời gian sống (TTL) và tự hết hạn, không ~~tốn tài nguyên cần~~ dọn dẹp thủ công, ~~và (iii).~~
- **Vô hiệu hóa chủ động:** việc ~~vô hiệu hóa chủ động xóa cache~~ chỉ là một lệnh xóa khóa, được ~~hệ thống~~ áp dụng ngay khi dữ liệu nguồn thay đổi (ví dụ xóa cache danh sách cửa hàng khi người dùng tạo cửa hàng mới) [22].

3.7 MinIO — ~~Object Storage~~ — Lưu trữ đối tượng

Để đáp ứng quy ~~Quy~~ trình quản lý ~~thông tin sản phẩm~~ ở ~~phần 2.3.3~~, hệ thống yêu cầu ~~mục 2.3.3~~ cần một ~~giải pháp nơi~~ lưu trữ tệp ~~tin hình ảnh sản phẩm định dạng nhị phân~~ theo ~~nhiều biến thể kích cỡ~~, đây là một môi trường dữ liệu, vốn không ~~thể dồn nén trực tiếp phù hợp để nhồi~~ vào ~~kho lưu trữ bảng biểu truyền thống cơ sở dữ liệu quan hệ~~.

Các ~~dịch vụ hỗ trợ cung cấp kho chứa~~ giải pháp lưu trữ đối tượng thay thế thường được tiếp cận là ~~(i) hệ sinh thái đám mây gồm AWS S3, (ii) không gian lưu trữ Cloudflare R2, và (iii) việc ghi tệp thẳng vào hệ thống lưu trữ tĩnh trên trực tiếp lên ổ đĩa máy chủ nội bộ.~~

MinIO được chọn vì ~~(i) sở hữu chuẩn giao thức giao tiếp y hệt như cấu trúc S3, giúp việc di dời toàn bộ ba lý do:~~

- **Tương thích S3:** có thể chuyển dữ liệu ~~hệ thống lên nền tảng trực tuyến lớn AWS S3 về sau mà gần như không gặp trở ngại gì sau này~~, ~~(ii) bản thân phần mềm cung cấp khả năng lưu trữ vận hành phải sửa mã.~~
- **Tự host được:** chạy ngay trên hạ tầng ~~đóng gói khép kín container~~, giúp ~~tiết kiệm kinh giảm chi phí tối đa, và (iii) khả năng ký ủy quyền ở giai đoạn đầu.~~
- **Hỗ trợ presigned URL:** cho phép trình duyệt tải ảnh lên trực tiếp ~~liên kết giúp trang mặt tiền có thể đưa dữ liệu lên mà không phải đi vòng qua máy chủ ảnh bỏ qua khâu xử lý luân chuyển ở trung tâm [23]. API [23].~~

3.8 Better Auth — ~~Xác thực và Phân phân quyền~~

Để đáp ứng yêu ~~Yêu~~ cầu bảo mật ở mục 2.4, ~~hệ thống cần một~~ cơ chế xác thực quản lý ~~được~~ hai loại người dùng tách biệt trên cùng hạ tầng: chủ cửa hàng đăng nhập vào bảng điều khiển và khách ~~mua~~ hàng đăng nhập vào trang cửa hàng, với phiên đăng nhập của hai nhóm không được lẫn vào nhau.

Các ~~công cụ thư viện~~ xác thực ~~khả dĩ cho giải pháp này bao thay thế~~ gồm ~~(i) thư~~

~~viện~~ NextAuth.js, ~~(ii)~~ Passport.js kết hợp quản lý phiên thủ công, và ~~(iii)~~ dịch vụ ~~xác thực~~ Clerk.

~~Thư viện~~ Better Auth được chọn vì ~~(i)~~ ba lý do:

- Phiên lưu trong cơ sở dữ liệu: phiên đăng nhập được lưu ~~trong cơ sở dữ liệu~~ qua Prisma, cho phép định nghĩa hai hệ phiên độc lập cho chủ cửa hàng và khách ~~mua~~ hàng với tiền tố cookie riêng, ~~(ii) xây dựng~~.
- An toàn kiểu: thư viện viết hoàn toàn bằng TypeScript nên kiểu dữ liệu ~~của~~ phiên được kiểm tra ngay khi biên dịch, và ~~(iii) tích hợp tự nhiên~~.
- Tích hợp đa nền tảng: ~~dùng được~~ với cả NestJS (qua Guard) lẫn Next.js (qua middleware) trong cùng monorepo [24].

3.9 Docker và Kubernetes — ~~Containerization~~ — Đóng gói và Orchestration ~~điều phối~~

~~Để đáp ứng yêu~~ Yêu cầu ~~phi chức năng~~ về độ tin cậy ~~được thể hiện tại 2.4~~, tất cả ở mục 2.4 ~~đòi hỏi~~ hệ ~~quản trị phụ trợ khổng lồ phải được giữ cho~~ thống vận hành đồng bộ và ổn định ~~ở mọi phiên bản qua nhiều lần~~ triển khai, ~~chưa kể đến yêu cầu bắt buộc phải~~ và sao lưu cơ sở dữ liệu tự động ~~định kỳ vào mỗi~~ hàng ngày ~~cuối~~ ngày.

Các phương pháp triển khai ~~quản lý cụm máy chủ truyền thống có thể kể đến như (i) thay thế gồm~~ cấu hình ~~trực tiếp~~ thủ công ~~bằng tiện ích hệ điều hành~~, ~~(ii) trình quản lý Docker Swarm tập trung, trên máy chủ, Docker Swarm~~ và ~~(iii) hệ thống phân phối của~~ Nomad.

~~Sự kết hợp giữa~~ Docker và Kubernetes được chọn ~~vì trong để dùng cho hai~~ giai đoạn ~~khác nhau~~. Ở giai đoạn phát triển, ~~hệ thống tự chứa cấp cho tốc độ khởi chạy tối đa, còn khi Docker đóng gói toàn bộ hạ tầng phụ trợ (PostgreSQL, MongoDB, Redis, MinIO) thành các container, giúp môi trường nhất quán giữa các máy~~. Ở giai đoạn vận hành ~~sản xuất thì (i) hệ sinh thái CronJob điều phối chuẩn xác, Kubernetes cung cấp ba khả năng cần thiết~~:

- CronJob: ~~chạy các tác vụ dọn dẹp và dự phòng theo kế hoạch~~, ~~(ii) chiến lược thay thế cuộn chiếu cho phép định kỳ như sao lưu cơ sở dữ liệu hàng ngày~~.
- Rolling update: cập nhật phiên bản ~~diễn ra hoàn toàn mà không làm ngưng trệ bất cứ tương tác nào~~, và ~~(iii) trình quản lý bí mật vô hiệu hóa hoàn toàn mọi rủi ro lộ lọt cấu hình riêng tư~~ [25], [26]. gián đoạn dịch vụ.
- Quản lý secret: tách thông tin nhạy cảm khỏi mã nguồn [25], [26].

~~Chương này đã làm rõ tất cả các~~ Bảng 3.1 tổng hợp toàn bộ công nghệ ~~được~~

Công nghệ	Phiên bản	Loại	Mục đích trong đề tài
Turborepo	2.9	Build Tool <u>tool</u>	Quản lý monorepo, bộ nhớ đệm xây dựng <u>build</u> tăng dần
NestJS	11.0	Backend Framework <u>framework</u>	REST API, đa thuê bao, DI <u>container</u> <u>dependency injection</u>
Next.js	16.2	Frontend Framework <u>framework</u>	Admin Dashboard + Storefront (SSR/RSC)
PostgreSQL	15	RDBMS	Lưu Shop, User, Order, Product (có cấu trúc)
Prisma	5.22	ORM	Truy vấn an toàn kiểu + di trú <u>migration</u> lược đồ
MongoDB	6	Document DB	Lưu GlobalLayout, PageLayout, catalog mẫu
Mongoose	8.2	ODM	Kiểm soát lược đồ tài liệu MongoDB
Redis	7	In-memory Cache <u>cache</u>	Đệm định danh tenant, trạng thái shop (TTL 300s)
MinIO + imgproxy	—	Object Storage <u>storage</u>	Lưu ảnh gốc <u>theo</u> S3 + biến đổi ảnh khi phân phối
Better Auth	1.6	Auth Library <u>library</u>	Phiên đăng nhập tách biệt owner/customer
Docker	—	Containerization	Môi trường phát triển nhất quán
Kubernetes	—	Orchestration	Triển khai sản xuất <u>vận hành</u> , CronJob sao lưu
Tailwind CSS	4.x	CSS Framework <u>framework</u>	Giao diện Admin Dashboard + Storefront
Zod	3.24	Validation	Kiểm tra dữ liệu vào/ra của API

Bảng 3.1: Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2

cùng vai trò của chúng. Hai lựa chọn để đáp ứng tương ứng từng yêu cầu cốt lõi từ Chương 2. Nhấn mạnh sự bổ sung linh hoạt và hợp lý giữa PostgreSQL trong quản trị các cấu trúc mang tính then chốt là việc dùng song song PostgreSQL cho dữ liệu cứng nhắc có cấu trúc và MongoDB trong xử lý thông tin mềm dẻo đa biến, cũng như sự đồng hành giữa Docker Compose lúc cho cấu hình giao diện linh hoạt, cùng việc tách Docker cho phát triển và Kubernetes ở giai đoạn cho vận hành thực tế. Trên cơ sở các công nghệ đã lựa chọn, Chương 4 trình bày thiết kế kiến trúc và kết quả triển khai hệ thống.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Trên cơ sở các yêu cầu đã phân tích ở Chương 2 và các công nghệ đã lựa chọn ở Chương 3, chương này trình bày quá trình hiện thực hóa hệ thống ShopVolo v2. Mục 4.1 mô tả thiết kế kiến trúc từ mức tổng quan tới mức chi tiết gói. Mục 4.2 đi sâu vào thiết kế giao diện, thiết kế lớp và thiết kế cơ sở dữ liệu. Mục 4.3 tổng hợp kết quả xây dựng ứng dụng, mục 4.4 trình bày quy trình kiểm thử, và mục 4.5 mô tả mô hình triển khai hệ thống.

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được tổ chức theo kiến trúc **Monorepo** quản lý bằng Turborepo, kết hợp tư tưởng **Modular Monolith** cho tầng backend. Kiến trúc Monorepo cho phép quản lý tập trung toàn bộ mã nguồn frontend, backend và các gói dùng chung trong một kho lưu trữ duy nhất, nhờ đó kiểu dữ liệu được chia sẻ xuyên suốt giữa các ứng dụng và quy trình xây dựng được tối ưu bằng bộ nhớ đệm tăng dần [16].

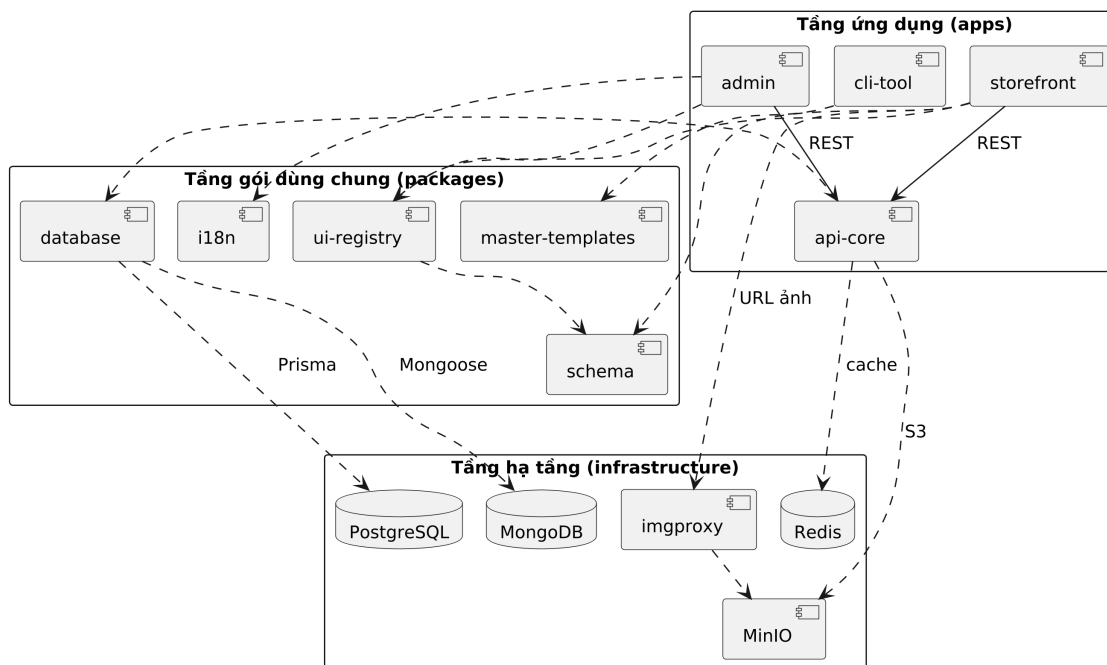
Hệ thống gồm ba nhóm thành phần chính. Nhóm thứ nhất là các ứng dụng: hai ứng dụng Next.js độc lập gồm `storefront` (mặt tiền cửa hàng phục vụ khách mua sắm) và `admin` (bảng điều khiển dành cho chủ cửa hàng), ứng dụng backend `api-core` viết bằng NestJS, và công cụ dòng lệnh `cli-tool` phục vụ vận hành. Nhóm thứ hai là năm gói thư viện nội bộ dùng chung: `database` (Prisma ORM và các mô hình Mongoose), `schema` (kiểu dữ liệu và xác thực Zod), `master-templates` (các bộ khung giao diện mẫu), `ui-registry` (thư viện thành phần giao diện kèm trình thiết kế kéo thả), và `i18n` (đa ngôn ngữ). Nhóm thứ ba là tầng hạ tầng gồm PostgreSQL, MongoDB, Redis, MinIO và imgproxy.

Bên trong `api-core`, mã nguồn được tổ chức thành 20 mô-đun nghiệp vụ (Auth, Shop, Order, Catalog, Layout, Inventory, Payment, Media, Promotions, Shipping, Wallet, Analytics, v.v.) theo đúng mô hình mô-đun của NestJS, tạo ranh giới rõ ràng giữa các miền nghiệp vụ và giữ khả năng tách thành các dịch vụ độc lập về sau.

Đối với bài toán đa thuê bao, hệ thống áp dụng chiến lược *Shared Database, Row-Level Isolation*: mọi thực thể nghiệp vụ đều mang khóa ngoại `shopId`, và việc phân lập logic được thực thi tự động thông qua `AsyncLocalStorage` ở tầng backend, bảo đảm mọi truy vấn đều gắn đúng ngữ cảnh cửa hàng mà không cần truyền tham số qua từng lớp. Chi tiết về cơ chế này được trình bày tại mục 5.1.

4.1.2 Thiết kế tổng quan

Hình 4.1 thể hiện biểu đồ gói của toàn hệ thống theo ba tầng. Nguyên tắc thiết kế xuyên suốt là phụ thuộc một chiều: tầng ứng dụng phụ thuộc vào tầng gói dùng chung, tầng gói dùng chung phụ thuộc vào tầng hạ tầng, và không tồn tại phụ thuộc ngược.



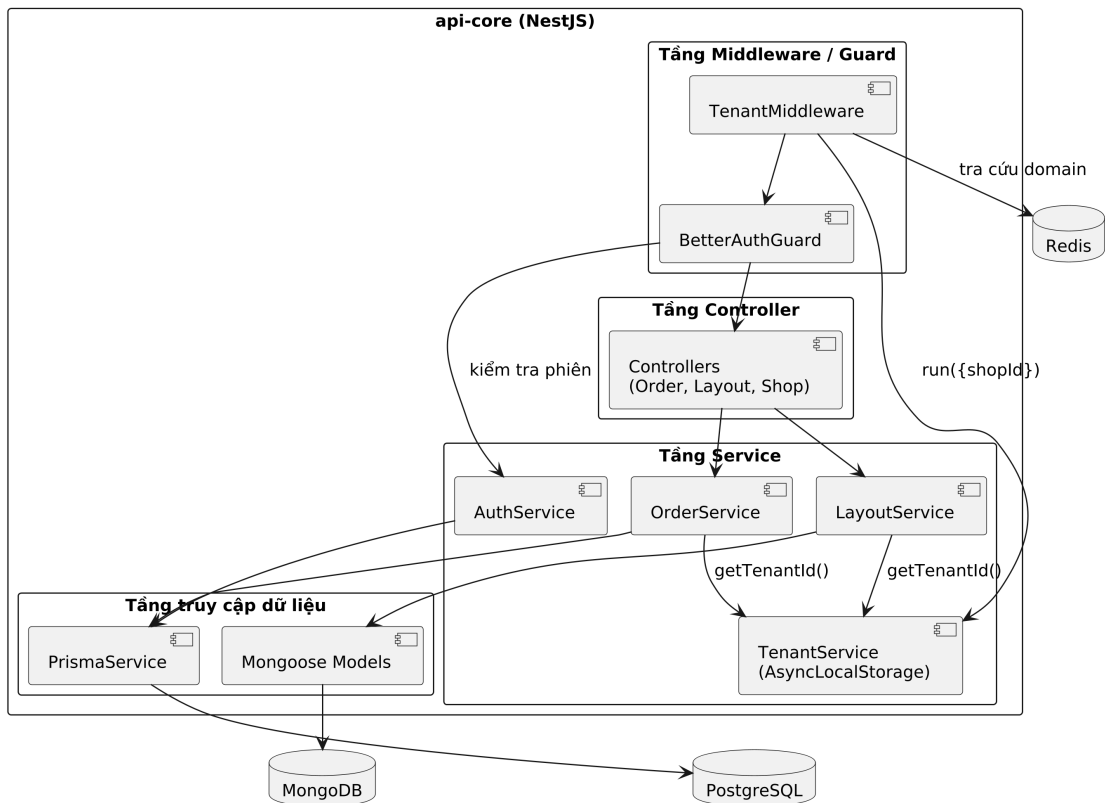
Hình 4.1: Biểu đồ gói thể hiện kiến trúc phân tầng và quan hệ phụ thuộc của hệ thống

Trong hình 4.1, hai ứng dụng admin và storefront cùng sử dụng ui-registry và i18n, đồng thời giao tiếp với api-core qua REST API; riêng storefront còn nhập kiểu dữ liệu từ schema và phân phối ảnh qua imgproxy. Gói ui-registry phụ thuộc schema để dùng chung định nghĩa UIComponentRef. Ứng dụng api-core truy cập PostgreSQL qua gói database (Prisma), truy cập MongoDB qua Mongoose, dùng Redis làm bộ nhớ đệm và MinIO làm kho lưu trữ đối tượng theo giao thức S3. Công cụ cli-tool thao tác với hệ thống qua REST API và sử dụng gói master-templates cho tác vụ đồng bộ giao diện mẫu.

4.1.3 Thiết kế chi tiết gói api-core

Bên trong api-core, mỗi request đi qua chuỗi xử lý phân tầng *Middleware* → *Guard* → *Controller* → *Service* → tầng truy cập dữ liệu, như thể hiện trong hình 4.2.

Theo hình 4.2, TenantMiddleware được đăng ký toàn cục và áp dụng cho mọi route: nó định danh cửa hàng từ header x-shop-id hoặc tên miền con, tra cứu Redis trước khi truy vấn PostgreSQL, rồi gọi `TenantService.run()`



Hình 4.2: Thiết kế chi tiết bên trong gói api-core và luồng xử lý đa thuê bao

để lưu ngữ cảnh vào AsyncLocalStorage. BetterAuthGuard bảo vệ các endpoint cần xác thực và nạp danh sách cửa hàng thuộc sở hữu của người dùng vào request. Các Controller chỉ làm nhiệm vụ tiếp nhận và xác thực dữ liệu vào, toàn bộ logic nghiệp vụ nằm ở tầng Service; các Service đọc định danh cửa hàng qua TenantService.getTenantId() rồi thao tác dữ liệu qua PrismaService (dữ liệu quan hệ) hoặc các mô hình Mongoose (tài liệu bố cục giao diện).

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Giao diện hệ thống được xây dựng theo hướng thành phần (component-based) với TailwindCSS phiên bản 4, trong đó toàn bộ thành phần tái sử dụng tập trung tại gói ui-registry. Ba quy chuẩn thiết kế được áp dụng thống nhất. Thứ nhất, về tính đáp ứng, giao diện hỗ trợ ba ngưỡng hiển thị: di động (dưới 768px), máy tính bảng (768–1024px) và máy tính để bàn (trên 1024px). Thứ hai, về giao diện động, trang Storefront không có bố cục cố định mà được kết xuất từ tài liệu JSON do api-core trả về, cho phép chủ cửa hàng tùy biến trang web bằng thao tác kéo thả; chi tiết cơ chế này được trình bày tại mục 5.2. Thứ ba, về phản hồi người dùng, mọi hành động đều có trạng thái đang tải và thông báo thành công hoặc lỗi rõ ràng;

ảnh sản phẩm hiển thị trước bằng ảnh mờ BlurHash trong lúc chờ tải ảnh thật, giúp bố cục không bị xô lệch.

Trình thiết kế kéo thả trong ứng dụng Admin là màn hình phức tạp nhất của hệ thống, gồm ba vùng: khung xem trước ở giữa (chuyển đổi được giữa chế độ máy tính và di động), bảng danh sách thành phần bên trái và bảng thuộc tính bên phải được sinh tự động từ lược đồ của từng thành phần. Hình ảnh chụp thực tế các màn hình sẽ được bổ sung tại mục 4.3.3.

4.2.2 Thiết kế lớp

Hình 4.3 trình bày thiết kế chi tiết các lớp giữ vai trò xương sống của `api-core`: `TenantService` và `TenantMiddleware` (cô lập đa thuê bao), `BetterAuthGuard` và `AuthService` (xác thực), `OrderService` (nghiệp vụ đặt hàng), `LayoutService` (quản lý bố cục giao diện) và `PrismaService` (truy cập dữ liệu).

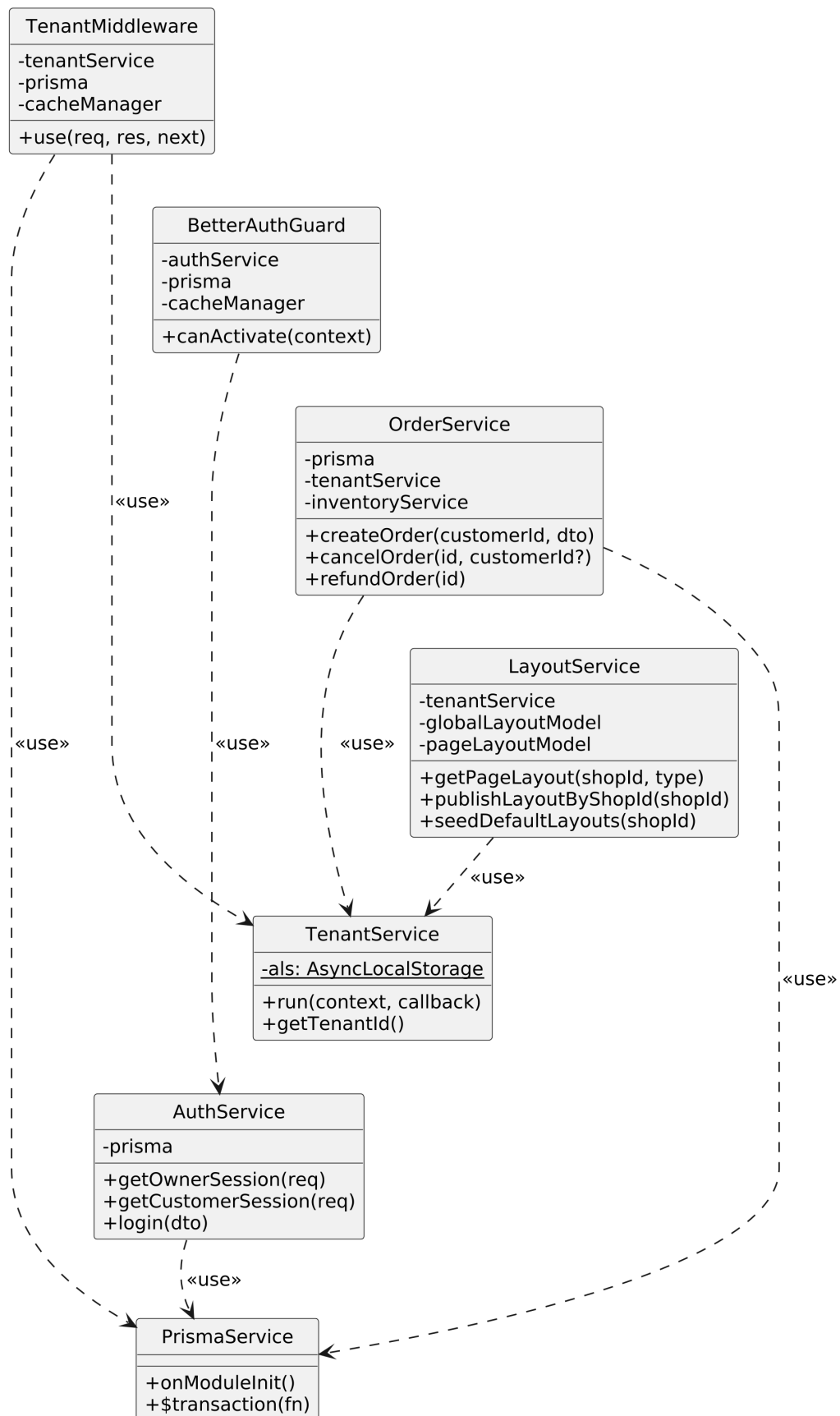
Trong hình 4.3, các quan hệ phụ thuộc đều hướng về hai lớp nền tảng: `TenantService` cung cấp ngữ cảnh cửa hàng cho mọi Service nghiệp vụ, còn `PrismaService` là điểm truy cập duy nhất tới PostgreSQL với khả năng thực thi giao dịch nguyên tử qua `$transaction`. `OrderService` tập trung nhiều phụ thuộc nhất vì nghiệp vụ đặt hàng phải phối hợp tồn kho, ví điện tử, thông báo và email.

Để làm rõ sự phối hợp giữa các lớp, hai luồng nghiệp vụ quan trọng nhất được mô tả bằng biểu đồ trình tự. Hình 4.4 thể hiện luồng đặt hàng: sau khi nạp giá các biến thể sản phẩm và kiểm tra mã khuyến mãi, toàn bộ thao tác ghi (trừ tồn kho, tạo đơn, tạo bản ghi thanh toán, xóa giỏ hàng, tạo vận đơn) được gói trong một giao dịch nguyên tử duy nhất; nếu thiếu tồn kho hoặc số dư ví không đủ, mọi thay đổi được hoàn tác toàn bộ.

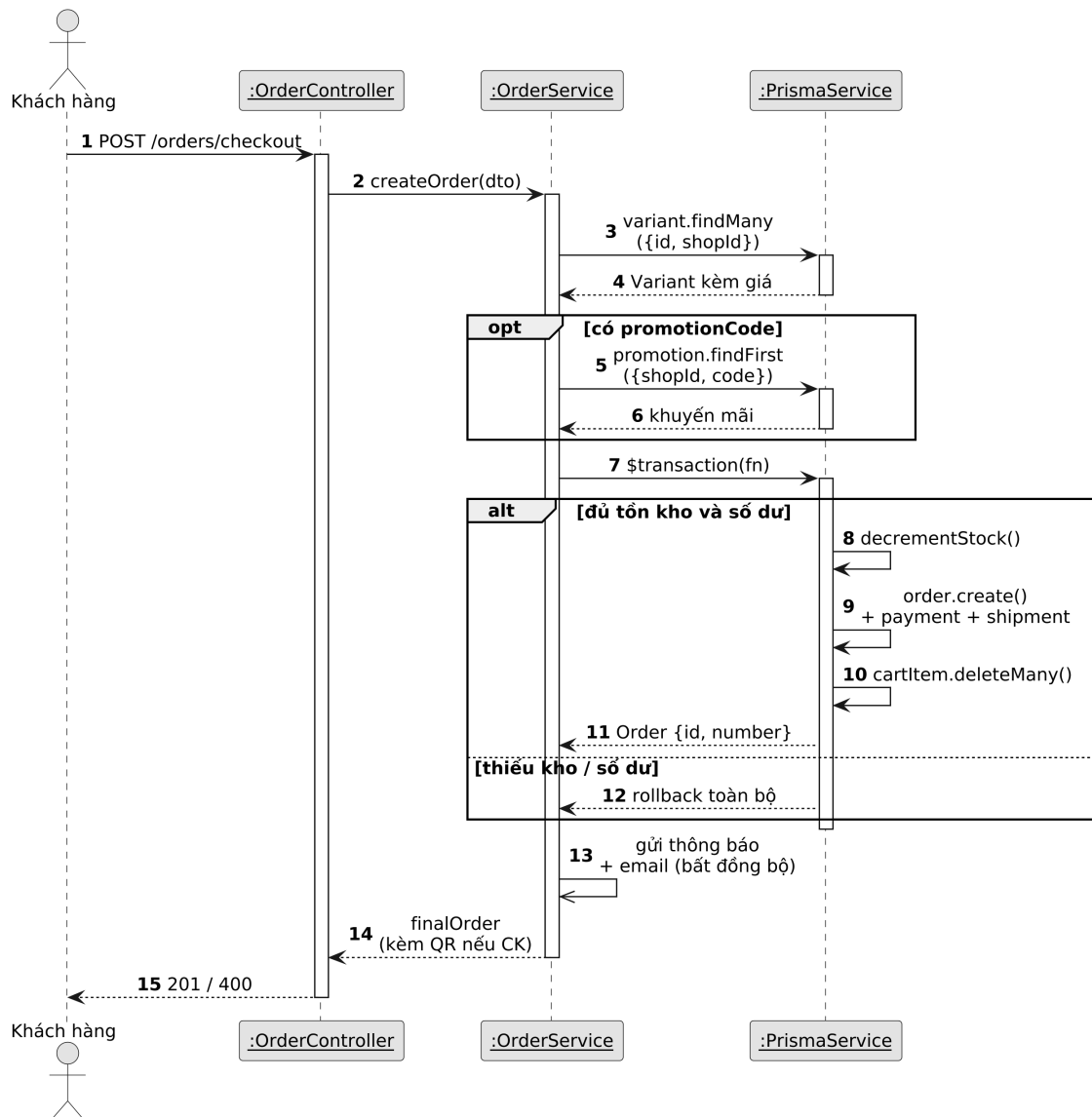
Hình 4.5 mô tả luồng xác thực tại `BetterAuthGuard`: các endpoint gắn `@Public()` được cho qua ngay; với endpoint cần bảo vệ, Guard xác minh phiên đăng nhập qua thư viện Better Auth theo đúng loại người dùng (chủ cửa hàng hoặc khách mua hàng, phân biệt bằng header `x-auth-type`), sau đó nạp danh sách cửa hàng thuộc sở hữu từ bộ nhớ đệm để phục vụ kiểm tra phân quyền ở các bước sau [24].

4.2.3 Thiết kế cơ sở dữ liệu

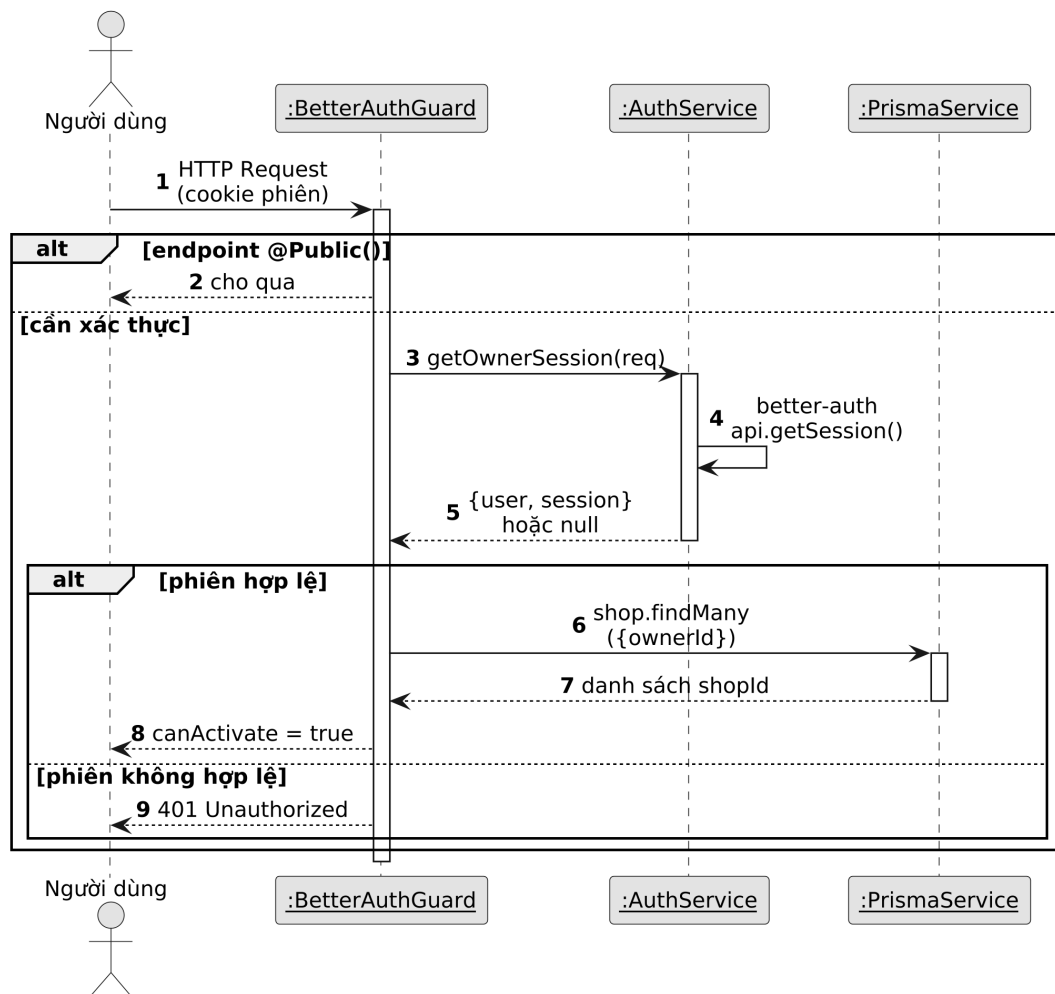
Hệ thống sử dụng đồng thời hai loại cơ sở dữ liệu theo đặc tính dữ liệu. PostgreSQL quản lý qua Prisma ORM lưu các thực thể có cấu trúc chặt và yêu cầu giao dịch (cửa hàng, sản phẩm, đơn hàng, thanh toán); lược đồ đầy đủ gồm



Hình 4.3: Biểu đồ lớp các thành phần cốt lõi của api-core

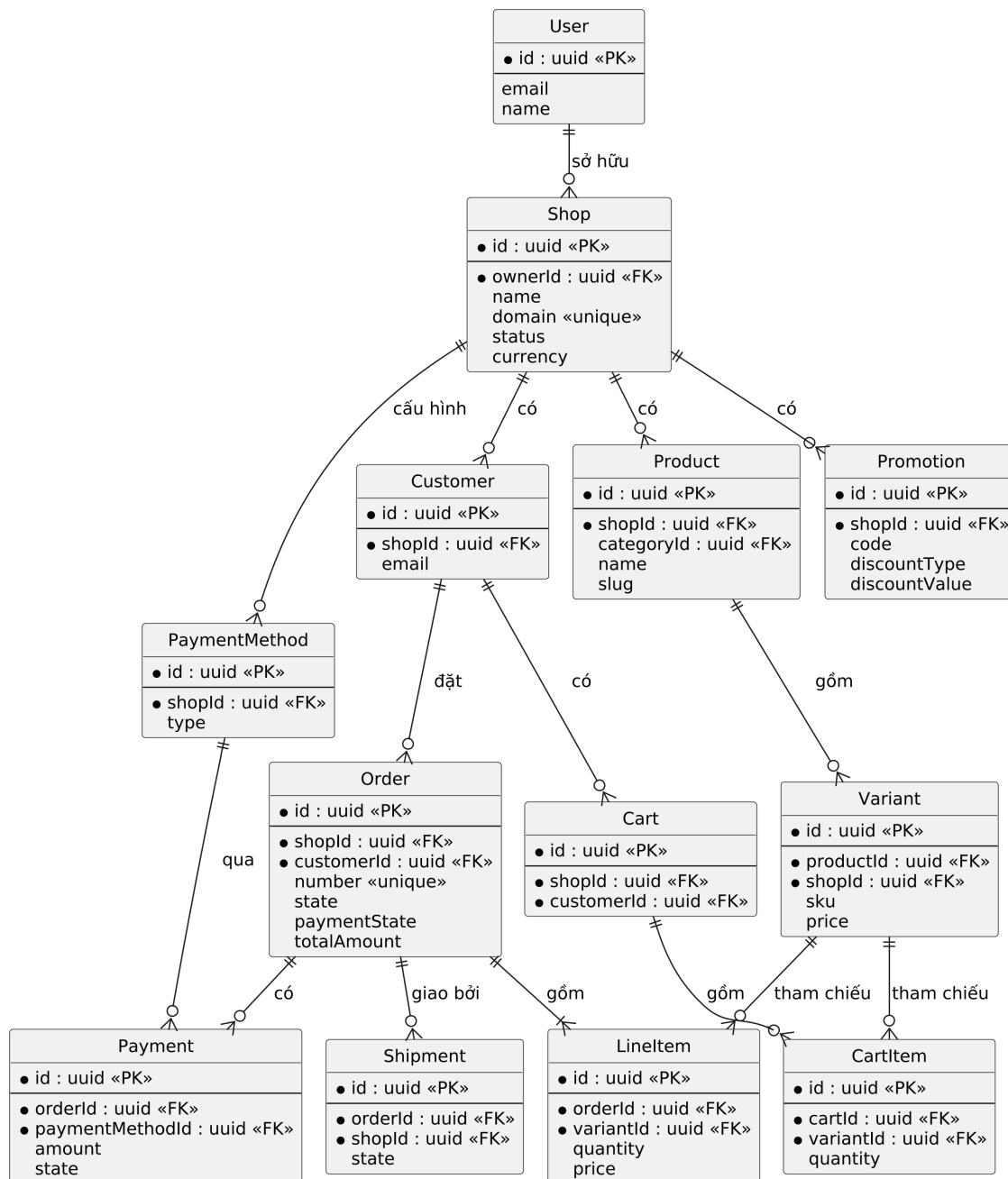


Hình 4.4: Biểu đồ trình tự luồng đặt hàng (checkout)



Hình 4.5: Biểu đồ trình tự luồng xác thực qua BetterAuthGuard

46-49 mô hình, trong đó hình 4.6 thể hiện các thực thể cốt lõi cùng quan hệ giữa chúng.



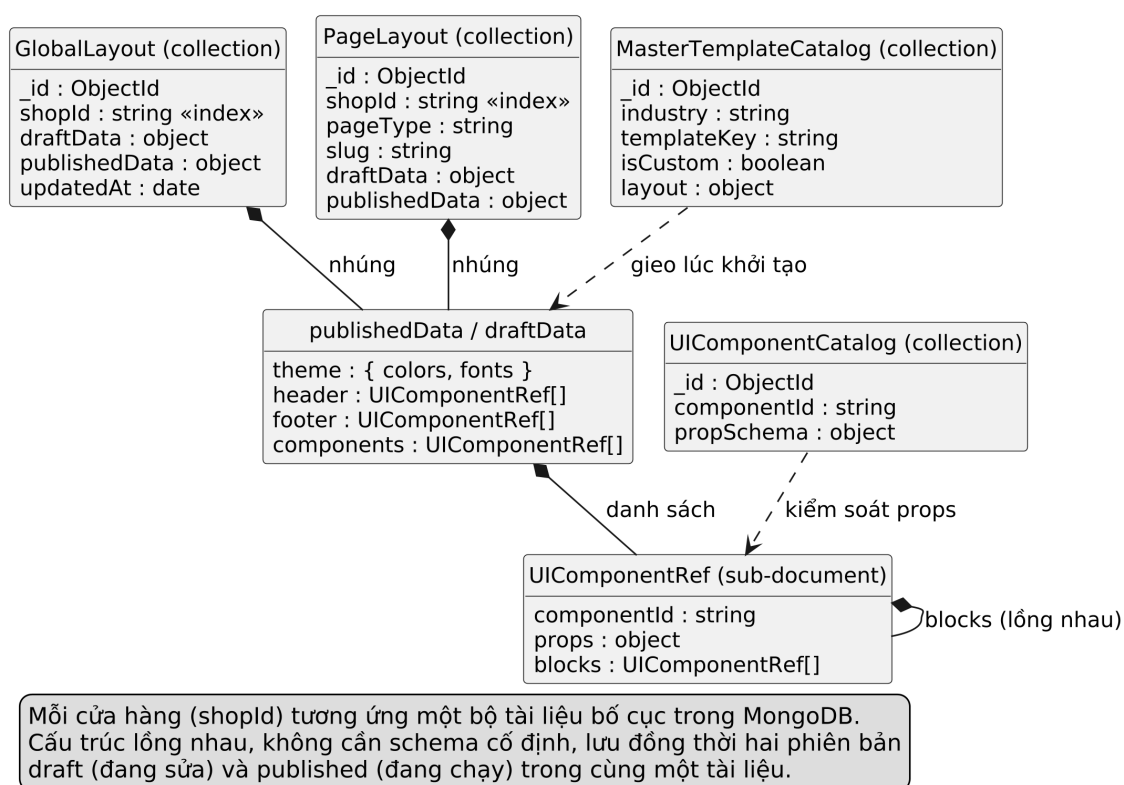
Hình 4.6: Sơ đồ thực thể liên kết các thực thể cốt lõi trên PostgreSQL

Hình 4.6 cho thấy đặc trưng đa thuê bao của lược đồ: hầu hết các thực thể đều mang khóa ngoại `shopId` trở về Shop, và các ràng buộc duy nhất đều được đặt trong phạm vi của hàng (ví dụ Variant duy nhất theo cặp (`shopId`, `sku`), Cart duy nhất theo cặp (`shopId`, `customerId`)). Nhờ vậy hai cửa hàng khác nhau có thể dùng trùng mã SKU mà không xung đột.

Bên cạnh PostgreSQL, hệ thống sử dụng ba kho dữ liệu chuyên biệt. MongoDB (qua Mongoose) lưu các tài liệu giao diện có cấu trúc linh hoạt:

GlobalLayout và PageLayout chứa hai phiên bản nháp và xuất bản của bố cục, UIComponentCatalog lưu lược đồ thuộc tính các thành phần giao diện, và MasterTemplateCatalog lưu danh mục giao diện mẫu theo ngành hàng. Redis lưu các kết quả định danh có tần suất truy cập cao: ánh xạ tên miền sang cửa hàng, trạng thái cửa hàng và danh sách cửa hàng thuộc sở hữu của người dùng, tất cả với thời gian sống 300 giây. MinIO lưu trữ tệp ảnh gốc theo giao thức S3, còn imgproxy biến đổi ảnh theo kích thước yêu cầu tại thời điểm phân phối.

Hình 4.7 mô tả cấu trúc các bộ sưu tập trên MongoDB. Khác với lược đồ quan hệ chuẩn hóa của PostgreSQL, các tài liệu giao diện được thiết kế lồng nhau (nested document) để khớp với cách dữ liệu được đọc và kết xuất. Điểm cốt lõi là hai trường draftData và publishedData trong GlobalLayout và PageLayout: cả hai cùng tham chiếu đề quy tới cấu trúc UIComponentRef – một cây thành phần giao diện trong đó mỗi nút chứa componentId, tập thuộc tính props và danh sách khối con blocks. Chính cấu trúc cây đề quy này là dữ liệu đầu vào cho động cơ kết xuất Zero-file được phân tích tại mục 5.2, còn cặp draftData/publishedData là nền tảng cho quy trình nháp–xuất bản hai pha tại mục 5.3.



Hình 4.7: Thiết kế tài liệu các bộ sưu tập giao diện trên MongoDB

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Mục đích	Công cụ / Thư viện	Phiên bản
Ngôn ngữ lập trình	TypeScript	5.7
Frontend framework	Next.js (App Router)	16.2
Thư viện giao diện	React	19.2
CSS framework	TailwindCSS	4.x
Backend framework	NestJS	11.0
ORM quan hệ	Prisma	5.22
Cơ sở dữ liệu quan hệ	PostgreSQL	15
Cơ sở dữ liệu tài liệu	MongoDB / Mongoose	6 / 8.2
Xác thực	Better Auth	1.6
Bộ nhớ đệm	Redis / ioredis	7 / 5.11
Hàng đợi tác vụ	Bull	4.16
Lưu trữ đối tượng	MinIO (AWS SDK v3)	—
Xử lý ảnh phân phối	imgproxy	—
Kiểm tra dữ liệu	Zod	3.24
Quản lý monorepo	Turborepo	2.9
Kiểm thử	Jest + @nestjs/testing	—

Bảng 4.1: Danh sách thư viện và công cụ sử dụng trong dự án

Bảng 4.1 liệt kê các thư viện và công cụ chính cùng phiên bản thực tế được khai báo trong tệp cấu hình của dự án.

4.3.2 Kết quả đạt được

Hệ thống đã hoàn thành việc xây dựng nền tảng thương mại điện tử đa thuê bao với đầy đủ các mô-đun cốt lõi. Bảng 4.2 thống kê các thông số kỹ thuật của mã nguồn (không tính mã kiểm thử, mã sinh tự động và thư viện bên thứ ba).

Thông số	Giá trị
Tổng số tệp mã nguồn TypeScript/TSX	391 tệp
Tổng số dòng mã nghiệp vụ	36.826 dòng
Số mô-đun nghiệp vụ trong api-core	20 mô-đun
Số lớp Controller / Service	24 / 27
Số gói thư viện nội bộ dùng chung	5 gói
Số mô hình dữ liệu PostgreSQL (Prisma)	46 49 mô hình
Số bộ sưu tập MongoDB chính	4 bộ sưu tập
Số thành phần giao diện trong ui-registry	hơn 40 thành phần
Số tệp kiểm thử tự động (Jest)	32 tệp

Bảng 4.2: Thống kê thông số kỹ thuật của dự án

Bảng 4.2 cho thấy quy mô mã nguồn ở mức một hệ thống hoàn chỉnh nhiều

phân hệ, trong đó tỷ lệ đáng kể nằm ở thư viện thành phần giao diện và các mô-đun nghiệp vụ backend.

4.3.3 Minh họa các chức năng chính

Phần này minh họa năm nhóm chức năng chính của hệ thống bằng ảnh chụp màn hình thực tế, bao gồm: ~~(i) luồng~~

- Luồng đăng ký và khởi tạo cửa hàng mới với bước chọn giao diện mẫu theo ngành hàng, ~~(ii) quản~~
- Quản lý sản phẩm và tồn kho trên bảng điều khiển Admin, ~~(iii) trình~~
- Trình thiết kế kéo thả tùy biến bố cục trang Storefront, ~~(iv) luồng~~
- Luồng mua hàng và thanh toán của khách, ~~và (v) theo~~
- Theo dõi, cập nhật trạng thái đơn hàng.

4.4 Kiểm thử

Hệ thống sử dụng Jest kết hợp @nestjs/testing để kiểm thử đơn vị và kiểm thử tích hợp cho tầng backend, với tổng cộng 32 tệp kiểm thử phủ các mô-đun nghiệp vụ chính (Order, Auth, Layout, Shop, Payment, Inventory, Promotions, Shipping, Wallet, Media, Analytics). Kỹ thuật trọng tâm là giả lập ~~qua tiêm (mock) các~~ phụ thuộc ~~(dependency injection mocking)~~ thông qua cơ chế dependency injection của NestJS: PrismaService, TenantService, các mô hình Mongoose và bộ nhớ đệm đều được ~~thay~~ inject bằng đối tượng giả, nhờ đó các bài kiểm thử chạy độc lập hoàn toàn với hạ tầng.

Chức năng được kiểm thử kỹ nhất là luồng đặt hàng OrderService.createOrder – nghiệp vụ phức tạp nhất hệ thống. Bảng 4.3 liệt kê các trường hợp kiểm thử tiêu biểu được trích từ bộ kiểm thử thực tế của mô-đun này.

Bảng 4.3 cho thấy bộ kiểm thử phủ cả luồng chính lẫn các luồng phát sinh của nghiệp vụ đặt hàng đã đặc tả tại mục 2.3.5. Ngoài ra, bộ kiểm thử của AuthService xác minh hành vi phiên đăng nhập (trả về phiên hợp lệ, trả về null khi phiên lỗi, đổi tên người dùng với các ràng buộc trùng tên), và bộ kiểm thử của LayoutService xác minh chu trình nhập–xuất bản (tự khởi tạo bố cục rỗng, ghi đè bản nháp, từ chối xuất bản khi chưa có nháp, sao chép nháp sang bản xuất bản). Toàn bộ các bài kiểm thử đều đạt khi chạy bằng lệnh `npm run test` thông qua Turborepo.

TC	Kịch bản	Kết quả mong đợi
1	Gọi nghiệp vụ khi thiếu ngữ cảnh tenant	Ném <code>BadRequestException</code> “Shop context is missing”
2	Checkout không kèm <code>lineItems</code> và giỏ hàng trống	Từ chối với lỗi 400
3	Checkout không kèm <code>lineItems</code> , giỏ hàng có hàng	Lấy giỏ phía máy chủ, đặt hàng và xóa giỏ trong cùng giao dịch
4	Variant không thuộc cửa hàng hiện tại	Từ chối với lỗi 400
5	Đặt hàng chuyển khoản ngân hàng	Tạo đơn trạng thái chờ kèm mã QR xác nhận
6	Thanh toán bằng ví đủ số dư	Đơn chuyển <code>confirmed/paid</code> , ví bị trừ đúng số tiền
7	Mã khuyến mãi phần trăm hợp lệ	Giảm đúng tỷ lệ, tăng bộ đếm lượt dùng
8	Mã khuyến mãi hết hạn hoặc vượt giới hạn lượt dùng	Từ chối với lỗi 400
9	Phương thức vận chuyển hợp lệ / không hợp lệ	Cộng đúng phí vận chuyển / từ chối với lỗi 400

Bảng 4.3: Các trường hợp kiểm thử tiêu biểu cho luồng đặt hàng

4.5 Triển khai

Hệ thống được thiết kế hướng cloud-native với hai cấu hình triển khai tương ứng hai giai đoạn. Trong giai đoạn phát triển, toàn bộ hạ tầng chạy bằng Docker Compose gồm các dịch vụ: hai thực thể PostgreSQL (chuẩn bị sẵn cho hướng phân mảnh dữ liệu về sau), MongoDB, MinIO kèm tác vụ khởi tạo bucket, và imgproxy; các ứng dụng Node.js chạy trực tiếp qua Turborepo để tận dụng khả năng nạp lại nhanh khi sửa mã.

Đối với môi trường vận hành, đồ án đã xây dựng đầy đủ bộ manifest Kubernetes gồm: ~~(i) các~~

- Các Deployment riêng cho ba ứng dụng `admin`, `api-core`, `storefront` với giới hạn tài nguyên tường minh cho từng container (ví dụ `api-core` giới hạn 350MB bộ nhớ, Redis giới hạn 512MB) nhằm ~~phục vụ mục tiêu vận hành trên hạ tầng khiêm tốn~~ giữ chi phí tài nguyên cho mỗi cửa hàng ở mức thấp như đã nêu tại mục 2.4; ~~(ii) tầng~~
- Tầng định tuyến gồm NGINX Ingress ánh xạ các tên miền dịch vụ và Cloudflare Tunnel cho phép công bố hệ thống ra Internet mà không cần địa chỉ IP công khai; ~~(iii) hạ~~
- Hạ tầng giám sát Grafana; ~~và (iv) một~~

- Một CronJob sao lưu PostgreSQL tự động chạy lúc 3 giờ sáng hằng ngày, đáp ứng yêu cầu độ tin cậy tại mục 2.4.

Tại thời điểm viết báo cáo, hệ thống đã vận hành ổn định trên môi trường phát triển; việc triển khai bộ manifest nói trên lên máy chủ thực tế và đo lường các chỉ tiêu hiệu năng là bước kế tiếp, như đã trình bày trong các mục kết quả của Chương 5 và định hướng tại mục 6.2.

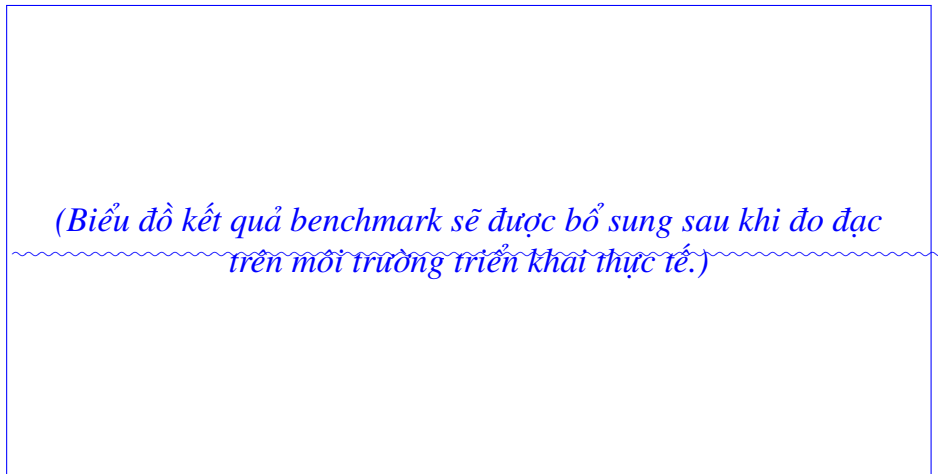
~~Chương này~~

4.5.1 Đánh giá hiệu năng (benchmark)

Để kiểm chứng mục tiêu chi phí biên thấp đã trình bày trọn vẹn quá trình đặt ra tại mục 2.4, đồ án thiết kế và xây dựng một quy trình đo hiệu năng (benchmark) chạy trên chính mã nguồn của hệ thống: từ kiến trúc. Quy trình gồm các bước:

- **Chuẩn bị dữ liệu:** dùng cli-tool sinh số lượng cửa hàng tăng dần (ví dụ 100, 1.000, 5.000 cửa hàng), mỗi cửa hàng kèm bố cục mặc định và một tập sản phẩm mẫu, để quan sát mức tiêu thụ tài nguyên thay đổi theo số cửa hàng.
- **Tạo tải:** dùng công cụ kiểm thử tải (k6 hoặc autocannon) bắn đồng thời vào các tên miền cửa hàng khác nhau, đo độ trễ ở phân vị p50/p95 cho hai trường hợp trùng cache và trượt cache.
- **Thu thập chỉ số:** ghi lại mức sử dụng RAM và CPU của api-core, Redis và Storefront qua hạ tầng Monorepo, thiết kế lớp giám sát Grafana, từ đó tính lượng bộ nhớ tăng thêm trung bình cho mỗi cửa hàng.
- **Chỉ số xây dựng:** đo thời gian build với và không có bộ nhớ đệm của Turborepo, cùng kích thước gói JavaScript của một trang Storefront để đánh giá hiệu quả của cơ sở dữ liệu, tối chế nạp thành phần theo nhu cầu.

Hình 4.8 là vị trí dành cho biểu đồ kết quả mã nguồn, kiểm thử tự động và mô hình triển khai. Một benchmark; số quyết định thiết kế mang tính đóng góp nổi bật mới chỉ được tóm lược ở đây; chương tiếp theo liệu sẽ phân tích chuyên sâu ba giải pháp kỹ thuật quan trọng nhất của đồ án. được bổ sung sau khi hệ thống được triển khai và đo đạc trên máy chủ thực tế theo quy trình nêu trên.



Hình 4.8: Biểu đồ kết quả đo hiệu năng theo số lượng cửa hàng

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày thiết kế tổng thể và kết quả xây dựng hệ thống ShopVolo v2. Chương 5 này đi sâu phân tích ba giải pháp kỹ thuật được xem là đóng góp nổi bật nhất của đề án, ~~bao gồm: (i) cơ~~:

- Cơ chế cô lập dữ liệu đa thuê bao tự động dựa trên `AsyncLocalStorage` ~~được trình bày tại~~ (mục 5.1, ~~(ii) kiến~~).
- Kiến trúc giao diện Zero-file cùng động cơ kết xuất động ~~được phân tích tại~~ (mục 5.2, ~~và (iii) mô~~).
- Mô hình Master Template với quy trình cá nhân hóa giao diện hai pha nhập–xuất bản ~~được mô tả tại~~ (mục 5.3).

Mỗi giải pháp được trình bày theo cấu trúc ba phần: dẫn dắt bài toán, chi tiết giải pháp, và kết quả đạt được. Cả ba giải pháp cùng hướng tới một mục tiêu chung là hạ chi phí biên của mỗi cửa hàng xuống mức tối thiểu trong khi vẫn giữ trải nghiệm dịch vụ tốt.

5.1 Cơ chế cô lập dữ liệu đa thuê bao dựa trên `AsyncLocalStorage`

5.1.1 Bài toán

Như đã phân tích tại mục 2.4, yêu cầu bảo mật quan trọng nhất của một nền tảng đa thuê bao theo chiến lược dùng chung cơ sở dữ liệu (`Shared Database`, `Row-Level Isolation`) là mọi truy vấn dữ liệu đều phải được ràng buộc theo định danh cửa hàng `shopId`. Chỉ cần một truy vấn duy nhất bỏ sót điều kiện này, dữ liệu của một cửa hàng có thể bị lộ sang cửa hàng khác.

Cách tiếp cận phổ biến là truyền tham số `shopId` một cách tường minh qua mọi tầng xử lý, từ `Controller` xuống `Service` rồi tới tầng truy cập dữ liệu. Với quy mô của hệ thống gồm 24 `Controller` và 27 `Service`, cách làm này khiến mọi chữ ký hàm đều phải mang thêm một tham số lặp lại, làm tăng khối lượng mã trùng lặp và đặc biệt là đặt toàn bộ trách nhiệm cô lập dữ liệu lên kỷ luật của người lập trình. Một hướng khác là sử dụng provider phạm vi request (`Scope.REQUEST`) của `NestJS`, tuy nhiên cơ chế này buộc framework khởi tạo lại toàn bộ chuỗi phụ thuộc cho từng request, gây tổn hao hiệu năng đáng kể trên các API có tần suất truy cập cao [17]. Hướng thứ ba là tách riêng lược đồ hoặc cơ sở dữ liệu cho từng cửa hàng; với mục tiêu ~~vận hành hàng chục nghìn~~ giữ chi phí biên thấp khi phục vụ nhiều cửa hàng ~~nhỏ~~ trên một hạ tầng ~~khíêm tốn~~ dùng chung, chi phí kết nối và quản trị tăng dần theo từng cửa hàng của hướng này là không ~~khả thi~~ phù hợp [27].

Bài toán đặt ra là: làm thế nào để mọi Service nghiệp vụ luôn biết được request hiện tại thuộc về cửa hàng nào mà không cần truyền tham số tường minh, đồng thời không phá vỡ mô hình singleton hiệu quả của container tiêm phụ thuộc dependency injection.

5.1.2 Giải pháp

Giải pháp của đề án khai thác AsyncLocalStorage – cơ chế lưu trữ ngữ cảnh theo chuỗi tác vụ bất đồng bộ của Node.js [28]. Đặc tính quan trọng của cơ chế này là ngữ cảnh được bảo toàn xuyên suốt toàn bộ chuỗi `await` của một request, và các request chạy đồng thời không thể đọc nhầm ngữ cảnh của nhau. Toàn bộ cơ chế được hiện thực hóa chỉ với hai thành phần đặt tại gói `common` của `api-core`.

Thành phần thứ nhất là lớp `TenantService` đóng vai trò bao bọc `AsyncLocalStorage`, với hai phương thức: `run(context, callback)` mở ra một vùng ngữ cảnh mới chứa `shopId`, và `getTenantId()` cho phép bất kỳ tầng nào đọc lại định danh đó. Mã nguồn 5.1 thể hiện toàn bộ phần lõi của cơ chế.

```

1 @Injectable()
2 export class TenantService {
3   private static readonly als = new AsyncLocalStorage<
4     TenantContext>();
5
6   run(context: TenantContext, callback: () => void) {
7     return TenantService.als.run(context, callback);
8   }
9
10  getTenantId(): string | undefined {
11    return TenantService.als.getStore()?.shopId;
12  }
13 }

```

Mã nguồn 5.1: Lớp `TenantService` bao bọc `AsyncLocalStorage`

Thành phần thứ hai là `TenantMiddleware` được đăng ký toàn cục, chịu trách nhiệm định danh cửa hàng cho từng request theo hai kênh: ~~(i) đọc~~

- Đọc trực tiếp header `x-shop-id` hoặc `x-tenant-id` do tầng frontend chuyển tiếp, ~~và (ii) nếu~~.
- Nếu không có header thì bóc tách tên miền con từ header `Host`, ví dụ `duck.myapp.com` cho ra định danh `duck`.

Việc ánh xạ từ tên miền sang `shopId` đòi hỏi truy vấn cơ sở dữ liệu, do đó kết quả được lưu vào Redis với khóa `domain:{subdomain}` trong 300 giây; chỉ khi trượt cache hệ thống mới truy vấn PostgreSQL. Middleware đồng thời kiểm tra trạng thái cửa hàng và chặn ngay các cửa hàng bị đình chỉ (SUSPENDED) với mã lỗi HTTP 403 trước khi request đi sâu vào hệ thống. Cuối cùng, middleware gọi `tenantService.run({shopId}, next)`, nhờ đó toàn bộ phần còn lại của chu trình xử lý request được thực thi bên trong vùng ngữ cảnh của đúng cửa hàng đó.

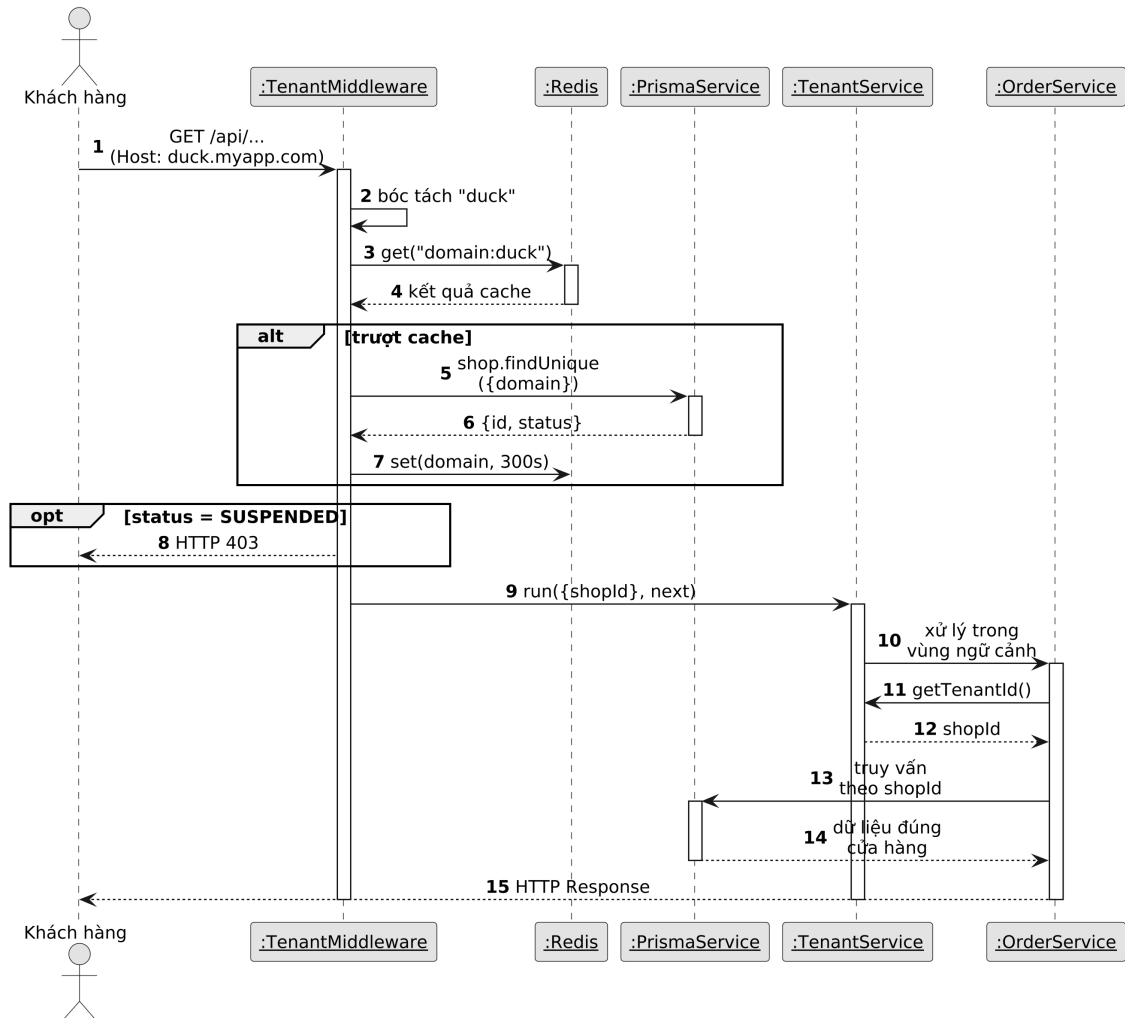
Ở phía sử dụng, mọi Service nghiệp vụ (`OrderService`, `ShopService`, `LayoutService`, v.v.) chỉ cần gọi một phương thức nội bộ `getShopId()`; phương thức này đọc định danh từ `TenantService` và chủ động ném ngoại lệ `BadRequestException` với thông điệp “Shop context is missing” nếu ngữ cảnh không tồn tại. Thiết kế “an toàn khi thiếu” (fail-safe) này đảo ngược rủi ro của cách truyền tham số tường minh: nếu người lập trình quên thiết lập ngữ cảnh, hệ thống từ chối phục vụ thay vì âm thầm trả về dữ liệu sai phạm vi.

Cơ chế trên được hỗ trợ bởi tầng xác thực: `BetterAuthGuard` sau khi xác minh phiên đăng nhập của chủ cửa hàng sẽ nạp danh sách các `shopId` mà người dùng đó sở hữu, lưu vào Redis với khóa `user:{userId}:shopIds` trong 300 giây. Khóa cache này bị xóa chủ động ngay khi người dùng tạo cửa hàng mới (trong `ShopService.createShop`), bảo đảm quyền sở hữu có hiệu lực tức thì thay vì phải chờ cache hết hạn. Hình 5.1 mô tả trình tự đầy đủ của quá trình định danh và truyền ngữ cảnh cho một request từ trang cửa hàng.

Hình 5.1 cho thấy hai nhánh xử lý của quá trình định danh: khi Redis đã có sẵn ánh xạ tên miền (cache hit), request không hề chạm tới PostgreSQL; ngược lại (cache miss), kết quả truy vấn được ghi ngược vào Redis để các request kế tiếp của cùng cửa hàng được phục vụ từ bộ nhớ đệm. Sau bước định danh, lời gọi `run()` bao trùm toàn bộ chuỗi Controller–Service–Prisma, do đó `OrderService` lấy được `shopId` mà không nhận bất kỳ tham số nào từ Controller.

5.1.3 Kết quả đạt được

Về mặt kiến trúc, toàn bộ 27 Service nghiệp vụ của `api-core` không có bất kỳ chữ ký hàm nào phải khai báo tham số `shopId` cho mục đích cô lập dữ liệu; logic định danh tenant tập trung tại đúng một điểm duy nhất là `TenantMiddleware`, giúp việc kiểm toán bảo mật chỉ cần tập trung vào một tệp mã nguồn thay vì rà soát từng truy vấn. Cơ chế cache hai lớp trên Redis (ánh xạ tên miền và trạng thái cửa hàng) giới hạn số truy vấn định danh tới PostgreSQL ở mức tối đa một lần mỗi 300 giây cho mỗi cửa hàng, bất kể lưu lượng truy cập.



Hình 5.1: Biểu đồ trình tự quá trình định danh tenant và truyền ngưỡng qua AsyncLocalStorage

~~Các~~ Về tài nguyên, do logic định danh tập trung tại middleware và không khởi tạo lại chuỗi phụ thuộc cho từng request (khác với hướng `Scope.REQUEST`), cơ chế này không làm tăng số ~~liệu~~ tiến trình hay bộ nhớ theo số cửa hàng. Các phép đo ~~lượng định lượng về độ trễ của bước định danh (trường hợp thực tế (trúng và trượt cache) cũng như chi phí phụ trội của AsyncLocalStorage sẽ được bổ sung sau khi hệ thống được triển khai chính thức trên máy chủ thực tế theo hiện theo quy trình benchmark mô hình đã mô tả tại mục 4.54.5.1.~~

5.2 Kiến trúc Zero-file và động cơ kết xuất giao diện động

5.2.1 Bài toán

Các nền tảng thương mại điện tử truyền thống quản lý giao diện theo hướng mỗi cửa hàng sở hữu một bộ giao diện (theme) gồm các tệp mã nguồn riêng. Cách tổ chức này kéo theo hai hệ quả khi vận hành ở quy mô lớn: ~~(i) mỗi~~

- ~~Mỗi~~ lần cửa hàng thay đổi giao diện, hệ thống phải biên dịch và triển khai lại tài nguyên riêng của cửa hàng đó, ~~và (ii) chi~~.
- ~~Chi~~ phí lưu trữ cùng bộ nhớ máy chủ tăng tuyến tính theo số cửa hàng, bởi mỗi cửa hàng chiếm một bộ tệp và một tiến trình phục vụ riêng.

Với mục tiêu ~~vận hành 20.000 giữ chi phí biên cho mỗi cửa hàng trên một hạ tầng 16GB RAM ở mức thấp~~ như đã đặt ra tại mục 2.4, mô hình mỗi cửa hàng một bộ tệp là không ~~khả thi phù hợp~~.

Bài toán đặt ra là thiết kế một cơ chế cho phép mỗi cửa hàng có giao diện hoàn toàn khác nhau về bố cục, màu sắc và nội dung, trong khi toàn hệ thống chỉ chạy đúng một ứng dụng Storefront duy nhất và việc thay đổi giao diện không sinh ra bất kỳ tệp mã nguồn hay lần biên dịch nào – nguyên lý mà đề án gọi là “Zero-file”.

5.2.2 Giải pháp

Giải pháp gồm ba khối hợp thành: biểu diễn giao diện bằng dữ liệu JSON, định tuyến đa thuê bao tại tầng biên, và động cơ kết xuất động phía máy chủ.

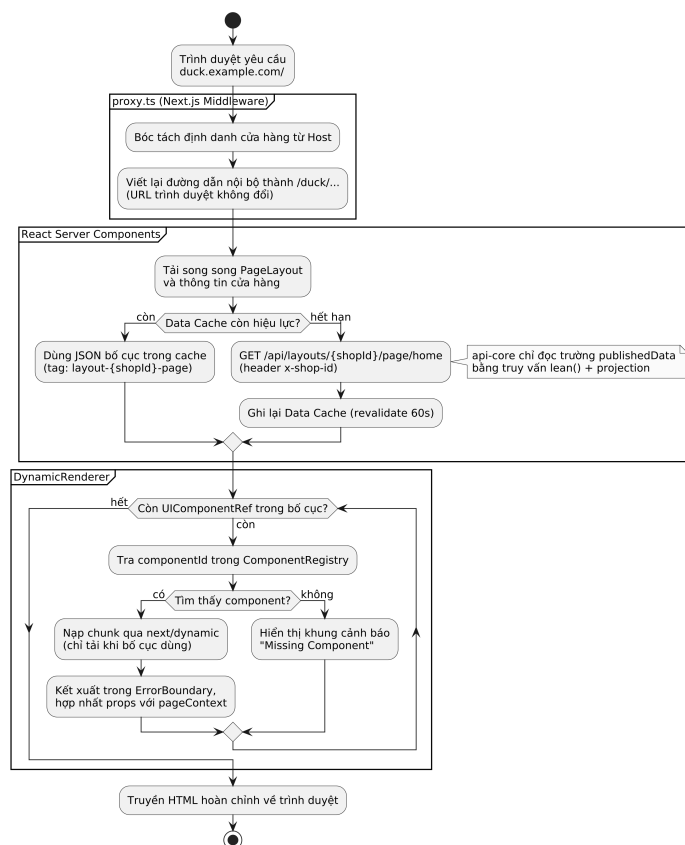
Thứ nhất, toàn bộ cấu trúc giao diện của một cửa hàng được biểu diễn bằng các tài liệu JSON lưu trong MongoDB: tài liệu `GlobalLayout` chứa các thành phần dùng chung (đầu trang, chân trang, bộ chủ đề màu sắc và phông chữ), còn mỗi tài liệu `PageLayout` mô tả một trang cụ thể theo `pageType` (trang chủ, danh sách sản phẩm, chi tiết sản phẩm). Mỗi thành phần giao diện trong tài liệu là một bản ghi `UIComponentRef` gồm ba thông tin cốt lõi: định danh loại thành phần `componentId`, tập thuộc tính hiển thị `props`, và danh sách khối con `blocks`. Nhờ biểu diễn này, “giao diện” của một cửa hàng chỉ là dữ liệu thuần túy, không gắn với bất kỳ tệp mã nguồn nào.

Thứ hai, tầng định tuyến đa thuê bao được hiện thực trong `proxy.ts` của ứng dụng Storefront bằng Next.js Middleware. Khi khách truy cập `duck.example.com/san-pham`, middleware bóc tách định danh cửa hàng từ tên miền và viết lại đường dẫn nội bộ thành `/duck/san-pham` mà không thay đổi URL trên trình duyệt. Cơ chế viết lại (rewrite) này cho phép một cây định tuyến App Router duy nhất với phân đoạn động `[shopSlug]` phục vụ mọi cửa hàng [18].

Thứ ba, động cơ kết xuất động hoạt động hoàn toàn trên máy chủ thông qua React Server Components. Khi nhận một request, trang RSC tải song song tài liệu bố cục và thông tin cửa hàng từ `api-core`, sau đó giao cho thành phần `DynamicRenderer` duyệt qua mảng `UIComponentRef`. ~~Trái tim~~ Thành phần cốt lõi của động cơ là `ComponentRegistry` – bảng ánh xạ từ chuỗi `componentId` sang React component thực tế trong gói `ui-registry`, hiện gồm hơn 40 thành phần được phân nhóm theo nghiệp vụ (banner, bộ sưu tập, sản phẩm, kể chuyện, văn bản và hai “mega section” cho trang danh mục và chi tiết sản phẩm). Mỗi mục trong bảng ánh xạ được khai báo qua `next/dynamic` với đường dẫn import sâu tới đúng tệp thành phần, nhờ đó mã của một thành phần chỉ được tải khi bố cục của cửa hàng thực sự sử dụng nó – tránh việc khách truy cập phải tải về toàn bộ thư viện giao diện. Ba cơ chế phòng vệ được cài sẵn trong động cơ: ~~thành-~~

- Thành phần không tồn tại trong bảng ánh xạ được thay bằng khung cảnh báo thay vì làm sập trang; ~~mỗi-~~
- Mỗi thành phần được bọc trong một `ErrorBoundary` riêng để lỗi kết xuất của một khối không lan sang phần còn lại; ~~và dữ-~~
- Dữ liệu nghiệp vụ thực (danh sách sản phẩm, chi tiết sản phẩm) được bơm vào qua `pageContext` tại thời điểm kết xuất, tách biệt hoàn toàn cấu hình giao diện khỏi dữ liệu.

Hiệu năng của kiến trúc được bảo đảm bằng chiến lược bộ nhớ đệm phân tầng theo độ biến động của dữ liệu: tài liệu bố cục được Next.js Data Cache lưu với chu kỳ làm mới 60 giây, dữ liệu sản phẩm 30 giây, thông tin cửa hàng 300 giây; mỗi mục cache được gắn nhãn (tag) theo cửa hàng và loại trang để có thể chủ động vô hiệu hóa khi chủ cửa hàng xuất bản giao diện mới. Ảnh sản phẩm không được xử lý trước theo nhiều kích cỡ mà được biến đổi tại thời điểm phân phối qua máy chủ `imgproxy`, với bộ nạp ảnh tùy biến sinh URL chứa tham số chiều rộng và chất lượng phù hợp với từng thiết bị. Hình 5.2 mô tả toàn bộ đường đi của một request từ trình duyệt tới HTML hoàn chỉnh.



Hình 5.2: Biểu đồ hoạt động luồng kết xuất giao diện động theo kiến trúc Zero-file

Hình 5.2 cho thấy điểm mấu chốt của kiến trúc: tại mọi bước từ định tuyến, tải bó cục đến kết xuất, hệ thống chỉ thao tác trên dữ liệu JSON và bảng ánh xạ trong bộ nhớ; không tồn tại bước đọc tệp giao diện riêng hay biên dịch lại mã nguồn cho từng cửa hàng.

5.2.3 Kết quả đạt được

Kiến trúc Zero-file đạt được mục tiêu thiết kế ở mức cấu trúc: việc khởi tạo thêm một cửa hàng mới không tạo ra bất kỳ tệp mã nguồn nào và không yêu cầu biên dịch hay triển khai lại; chi phí biên của mỗi cửa hàng chỉ còn là vài tài liệu JSON trong MongoDB. Một ứng dụng Storefront duy nhất với hơn 40 thành phần giao diện trong ui-registry phục vụ được không gian tùy biến lớn (tổ hợp tùy ý các thành phần trên ba loại trang cùng bộ chủ đề riêng), trong khi cơ chế import sâu kết hợp next/dynamic giữ cho dung lượng JavaScript của mỗi trang tỷ lệ với số thành phần thực dùng thay vì với kích thước toàn thư viện.

Chính nhờ đặc tính “một ứng dụng phục vụ mọi cửa hàng” này mà chi phí bộ nhớ trên mỗi cửa hàng được giữ ở mức rất thấp do không sinh tiến trình riêng cho từng cửa hàng. Các phép đo ~~định lượng về~~ thời gian phản hồi khi trùng và trượt cache, cùng mức tiêu thụ bộ nhớ ~~khi mô phỏng theo~~ số ~~lượng lớn~~ cửa hàng, sẽ được thực hiện ~~và bổ sung sau khi hệ thống được triển khai trên máy chủ thực tế~~

như [theo quy trình bày benchmark](#) tại mục [4.54.5.1](#).

5.3 Mô hình Master Template và quy trình cá nhân hóa giao diện hai pha

5.3.1 Bài toán

Kiến trúc Zero-file tại mục 5.2 trả lời câu hỏi “hệ thống kết xuất giao diện từ dữ liệu như thế nào”, nhưng để nền tảng dùng được bởi người không biết lập trình thì còn phải trả lời hai câu hỏi tiếp theo. Thứ nhất, dữ liệu bố cục ban đầu của một cửa hàng mới đến từ đâu – không thể bắt một chủ cửa hàng mới mở phải tự lắp ráp từng khối giao diện từ trang trắng. Thứ hai, làm thế nào để chủ cửa hàng chỉnh sửa giao diện một cách an toàn – nếu mọi thao tác kéo thả đều có hiệu lực tức thì trên trang đang bán hàng, khách truy cập sẽ nhìn thấy giao diện dở dang trong suốt quá trình chỉnh sửa.

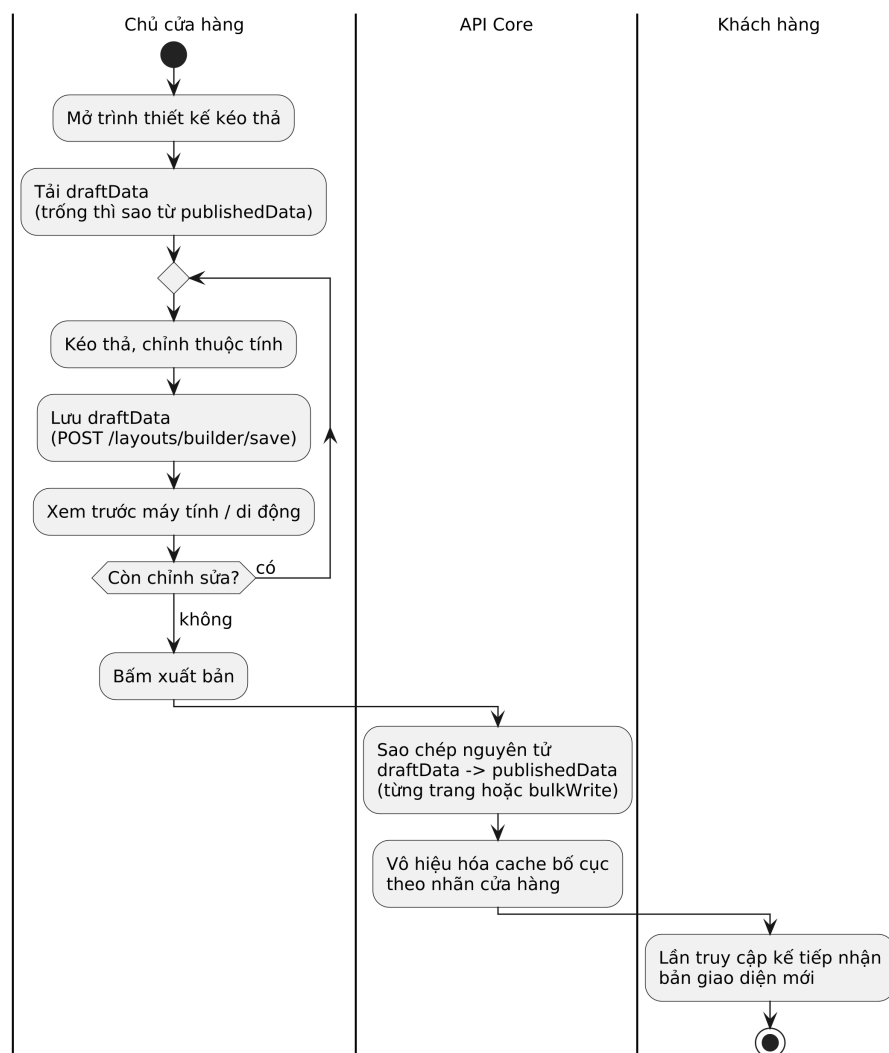
5.3.2 Giải pháp

Đối với câu hỏi thứ nhất, đồ án xây dựng cơ chế Master Template gồm hai phần. Phần tĩnh là gói `master-templates` chứa các bộ khung giao diện mẫu được thiết kế sẵn theo phong cách trình bày (chuẩn mực, thiên về hình ảnh, thiên về thông số kỹ thuật, và dịch vụ); mỗi bộ khung là một tài liệu JSON hoàn chỉnh gồm bộ chủ đề (bảng màu, kiểu chữ), các thành phần dùng chung và danh sách thành phần cho từng trang. Phần động là bộ sưu tập `MasterTemplateCatalog` trong MongoDB liệt kê các mẫu theo ngành hàng (thời trang, điện tử, mẹ và bé, thể thao, thực phẩm đóng gói, v.v.) để hiển thị trong bước chọn mẫu của quy trình khởi tạo cửa hàng. Khi chủ cửa hàng hoàn tất biểu mẫu khởi tạo, ứng dụng Admin gọi tuần tự hai API: tạo bản ghi cửa hàng trong PostgreSQL, sau đó gieo bố cục khởi điểm qua `POST /api/layouts/{shopId}/seed`. Thao tác gieo được thiết kế lũy đẳng (idempotent): hệ thống chỉ ghi dữ liệu mặc định vào những tài liệu còn trống (kiểm tra qua hàm `hasContent`), do đó việc gọi lại API này không bao giờ ghi đè lên các chỉnh sửa đã có của chủ cửa hàng. Nhờ vậy, ngay sau bước khởi tạo, cửa hàng đã có đầu trang, chân trang cùng ba trang khởi điểm hoạt động được, thay vì một trang trắng hay thông báo lỗi thiếu bố cục.

Đối với câu hỏi thứ hai, đồ án áp dụng mô hình dữ liệu hai pha: mỗi tài liệu bố cục (cả `GlobalLayout` lẫn `PageLayout`) chứa đồng thời hai phiên bản – `draftData` là bản nháp đang chỉnh sửa và `publishedData` là bản đã xuất bản. Hai phiên bản này phân tách triệt để hai phía của hệ thống: trình thiết kế kéo thả trong Admin chỉ đọc và ghi `draftData` (khi bản nháp trống thì sao lưu từ bản đã xuất bản để chỉnh tiếp), còn Storefront công khai chỉ đọc `publishedData` thông qua truy vấn `lean()` kèm phép chiếu trường (projection) nhằm giảm tối đa chi phí trên đường dẫn nóng nhất của hệ thống. Thao tác xuất bản là một phép sao chép

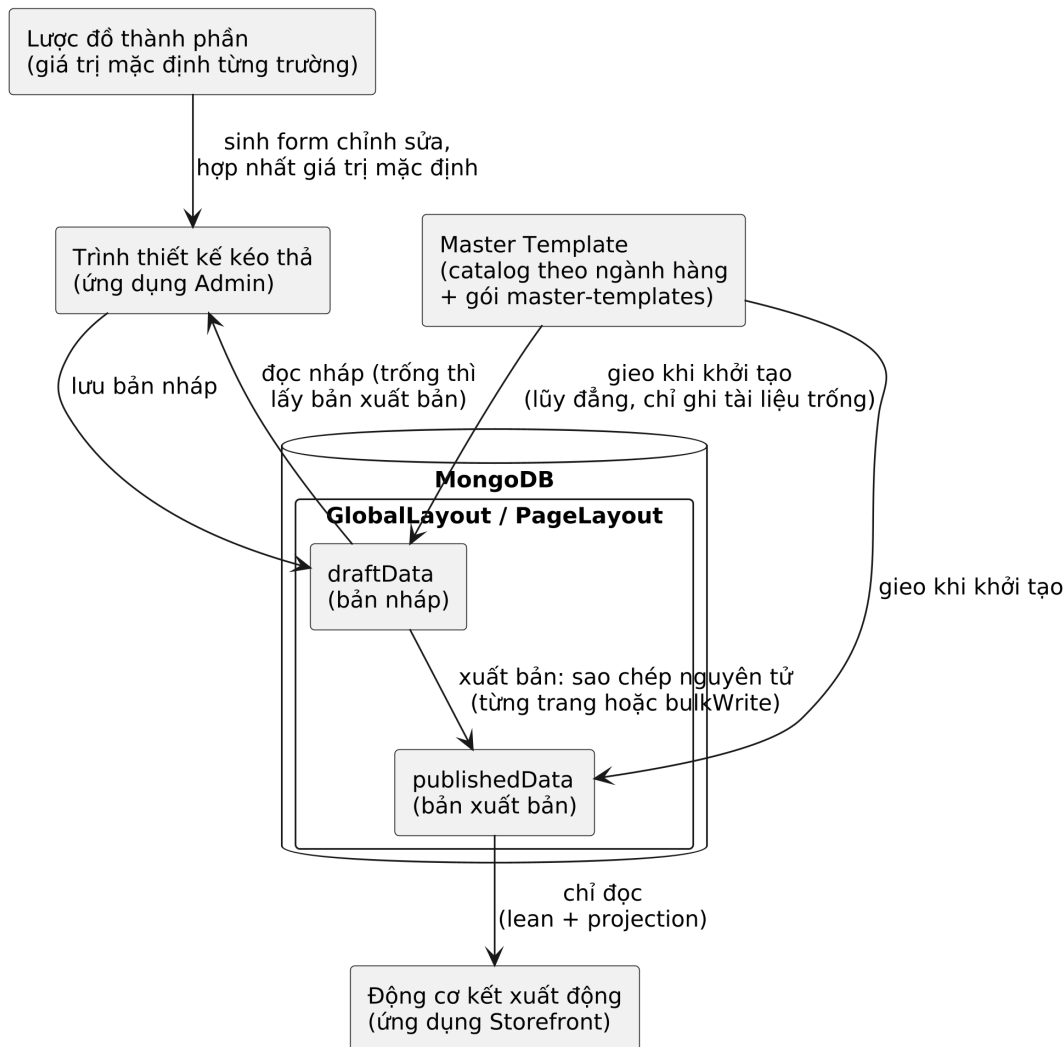
nguyên tử `draftData` → `publishedData`, được cung cấp ở hai mức hạt: xuất bản từng trang riêng lẻ phục vụ trình hướng dẫn từng bước (chủ cửa hàng hoàn thiện trang nào thì trang đó lên sóng ngay), và xuất bản toàn bộ bằng một lệnh `bulkWrite` duy nhất gom mọi trang trong một vòng giao tiếp tới MongoDB. Trình thiết kế phía Admin được xây dựng theo hướng schema-driven: mỗi loại thành phần khai báo một lược đồ thuộc tính (kiểu trường, nhãn hiển thị, giá trị mặc định, các khối con được phép), từ đó giao diện chỉnh sửa được sinh tự động và giá trị mặc định của lược đồ được hợp nhất với phần tùy biến của chủ cửa hàng tại thời điểm thêm thành phần.

Hình 5.3 mô tả luồng thực tế của quy trình hai pha: từ thao tác kéo thả của chủ cửa hàng (đầu vào), qua cơ chế lưu bản nháp và xuất bản nguyên tử ở hạ tầng, tới thời điểm khách truy cập nhận được giao diện mới (đầu ra). Vòng lặp chỉnh sửa chỉ tác động vào `draftData`, do đó trang đang bán hàng không bị ảnh hưởng cho tới khi chủ cửa hàng chủ động bấm xuất bản.



Hình 5.3: Biểu đồ hoạt động quy trình chỉnh sửa nháp và xuất bản giao diện hai pha

Tổng hợp lại, cấu hình giao diện mà khách truy cập nhìn thấy là kết quả hợp nhất của bốn tầng kế thừa lần lượt: giá trị mặc định trong lược đồ thành phần, bộ khung Master Template được gieo lúc khởi tạo, các chỉnh sửa nháp của chủ cửa hàng, và bản xuất bản cuối cùng; tầng sau ghi đè tầng trước ở đúng phạm vi nó thay đổi. Hình 5.4 minh họa mô hình phân tầng này cùng chu trình nháp–xuất bản.



Hình 5.4: Mô hình hợp nhất cấu hình giao diện phân tầng và chu trình nháp–xuất bản

Hình 5.4 cho thấy hai dòng chảy tách biệt quanh kho dữ liệu bố cục: dòng chỉnh sửa của chủ cửa hàng chỉ tác động vào bản nháp, còn dòng phục vụ khách truy cập chỉ đọc bản xuất bản; điểm giao duy nhất giữa hai dòng là thao tác xuất bản có chủ đích của chủ cửa hàng.

5.3.3 Kết quả đạt được

Mô hình Master Template kết hợp quy trình hai pha hoàn thiện trải nghiệm khởi tạo cửa hàng mô tả tại mục 2.3.1: chủ cửa hàng đi từ biểu mẫu đăng ký đến một trang web hoàn chỉnh có thể truy cập được mà không cần viết bất kỳ dòng mã nào, sau đó tùy biến dần qua trình hướng dẫn tám bước (tạo cửa hàng, thêm sản phẩm,

tạo bộ sưu tập, thiết lập menu, thiết kế trang chủ giao diện, cấu hình thanh toán, và vận chuyển và xác minh tên miền). Cơ chế gieo lũy đăng loại bỏ hoàn toàn lớp lỗi “trang trắng” cho cửa hàng mới, trong khi mô hình nháp–xuất bản bảo đảm khách truy cập không bao giờ nhìn thấy trạng thái chỉnh sửa dở dang. Về hiệu năng ghi, thao tác xuất bản toàn bộ gom mọi trang vào một lệnh bulkWrite duy nhất; về hiệu năng đọc, đường phục vụ khách chỉ tải về trường publishedData cần thiết.

Số liệu thống kê về dung lượng trung bình của tài liệu bố cục trên mỗi cửa hàng – cơ sở để ước lượng chi phí lưu trữ biên khi nhân rộng số lượng cửa hàng – sẽ được thu thập và bổ sung sau khi hệ thống vận hành trên môi trường triển khai thực tế.

Chương này đã phân tích ba đóng góp kỹ thuật trọng tâm của đề án: cơ chế cô lập dữ liệu đa thuê bao tự động qua AsyncLocalStorage, kiến trúc giao diện Zero-file với động cơ kết xuất động, và mô hình Master Template cùng quy trình cá nhân hóa hai pha. Ba giải pháp này lần lượt giải quyết ba bài toán cốt lõi đã đặt ra từ Chương 1: an toàn dữ liệu giữa các cửa hàng, chỉ là một số tài liệu JSON bố cục thay vì một bộ tệp theme riêng, góp phần trực tiếp vào mục tiêu chi phí biên gần như bằng không khi nhân rộng số cửa hàng, và khả năng tùy biến sâu dành cho người không biết lập trình. Chương tiếp theo tổng kết các kết quả đã đạt được thấp của toàn bộ đề án và đề xuất các hướng phát triển tiếp theo hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã hoàn thành mục tiêu xây dựng ShopVolo v2 – một nền tảng thương mại điện tử đa thuê bao cho phép các doanh nghiệp vừa và nhỏ khởi tạo cửa hàng trực tuyến mà không cần kỹ năng lập trình. Hệ thống được tổ chức dưới dạng Monorepo gồm bốn ứng dụng (Admin Dashboard, API Core, Storefront và công cụ dòng lệnh vận hành) cùng năm gói thư viện dùng chung, hiện thực hóa đầy đủ mười nhóm chức năng đã xác định tại mục 2.1, từ khởi tạo cửa hàng, quản lý sản phẩm và tồn kho, tùy biến giao diện kéo thả, cho tới giỏ hàng, đặt hàng và quản lý vòng đời đơn hàng.

So với các sản phẩm tương tự đã khảo sát tại mục 2.1, ShopVolo v2 định vị ở điểm giao mà các nền tảng hiện có chưa đáp ứng đồng thời. Khác với Shopify và Haravan vốn giới hạn người dùng trong các bộ giao diện dựng sẵn, hệ thống cho phép tùy biến cấu trúc trang ở mức từng khối thành phần thông qua trình thiết kế kéo thả mà không yêu cầu viết mã. Khác với WooCommerce vốn đòi hỏi mỗi cửa hàng tự vận hành hạ tầng riêng, toàn bộ cửa hàng trên ShopVolo v2 chia sẻ một hạ tầng duy nhất với chi phí biên cho mỗi cửa hàng mới chỉ là vài tài liệu JSON trong cơ sở dữ liệu. Cần lưu ý rằng phép so sánh này dừng ở mức kiến trúc và tính năng; việc đối chứng định lượng về hiệu năng với các nền tảng thương mại sẽ chỉ có ý nghĩa sau khi hệ thống được đo đạc trên môi trường triển khai thực tế.

Về mặt kỹ thuật, đóng góp của đồ án tập trung ở ba giải pháp đã phân tích trong Chương 5: cơ chế cô lập dữ liệu đa thuê bao tự động bằng `AsyncLocalStorage` giúp loại bỏ hoàn toàn việc truyền định danh cửa hàng qua các tầng xử lý, kiến trúc giao diện Zero-file cho phép một ứng dụng duy nhất kết xuất giao diện khác nhau cho mọi cửa hàng từ dữ liệu JSON, và mô hình Master Template với quy trình nháp–xuất bản hai pha bảo đảm trải nghiệm chỉnh sửa an toàn cho người không chuyên. Bên cạnh đó, quá trình xây dựng hệ thống cũng tạo ra một nền tảng mã nguồn có cấu trúc rõ ràng với 24 Controller, 27 Service được kiểm thử bằng Jest, và lược đồ cơ sở dữ liệu 46 thực thể phủ trọn nghiệp vụ bán lẻ trực tuyến.

Bên cạnh các kết quả trên, đồ án còn một số hạn chế. Thứ nhất, việc tích hợp cổng thanh toán trực tuyến của bên thứ ba mới dừng ở mức mô phỏng; các phương thức thanh toán đang vận hành thực tế là thanh toán khi nhận hàng, chuyển khoản ngân hàng xác nhận qua mã QR và ví điện tử nội bộ của nền tảng. Thứ hai, các chỉ tiêu hiệu năng đặt ra tại mục 2.4 (thời gian phản hồi, số cửa hàng phục vụ đồng thời trên một cấu hình phần cứng) chưa được kiểm chứng bằng số liệu đo lường do

hệ thống chưa được triển khai lên máy chủ sản xuất. Thứ ba, một số thành phần giao diện trong thư viện `ui-registry` mới ở dạng khung chờ hoàn thiện, và hạ tầng cơ sở dữ liệu tuy đã chuẩn bị sẵn hai phân mảnh PostgreSQL nhưng tầng ứng dụng chưa hiện thực logic định tuyến phân mảnh.

Quá trình thực hiện đồ án mang lại nhiều bài học có giá trị. Việc thiết kế theo nguyên tắc “an toàn khi thiếu” – hệ thống từ chối phục vụ khi thiếu ngữ cảnh thay vì âm thầm chạy sai – cho thấy hiệu quả rõ rệt trong việc ngăn lỗi rò rỉ dữ liệu ngay từ giai đoạn phát triển. Kỷ luật vô hiệu hóa bộ nhớ đệm chủ động (xóa cache đúng lúc dữ liệu nguồn thay đổi) quan trọng không kém việc thiết lập bộ nhớ đệm. Cuối cùng, mô hình Monorepo với các gói kiểu dữ liệu dùng chung giữa frontend và backend giúp phát hiện phần lớn lỗi không tương thích ngay tại thời điểm biên dịch thay vì lúc vận hành.

6.2 Hướng phát triển

Trong thời gian tới, ưu tiên trước hết là hoàn thiện các chức năng hiện có. Hệ thống cần được triển khai lên máy chủ thực tế bằng Docker theo mô hình đã thiết kế tại mục 4.5, từ đó tiến hành đo lường đầy đủ các chỉ tiêu hiệu năng còn để ngỏ trong Chương 5 (độ trễ định danh tenant, thời gian phản hồi khi trúng và trượt cache, mức tiêu thụ bộ nhớ theo số cửa hàng) và đối chiếu với các mục tiêu phi chức năng tại mục 2.4. Song song với đó, mô-đun thanh toán cần được nâng cấp từ mức mô phỏng lên tích hợp thật với các cổng thanh toán nội địa như VNPay và MoMo, bao gồm việc hiện thực đầy đủ cơ chế webhook có xác minh chữ ký số để tự động đối soát giao dịch. Các thành phần giao diện còn ở dạng khung trong `ui-registry` cũng cần được hoàn thiện để mở rộng không gian tùy biến cho chủ cửa hàng.

Về các hướng đi mới, kiến trúc hiện tại mở ra một số khả năng nâng cấp tự nhiên. Hướng thứ nhất là hiện thực logic định tuyến phân mảnh ở tầng ứng dụng nhằm khai thác hai phân mảnh PostgreSQL đã chuẩn bị sẵn trong hạ tầng, cho phép mở rộng theo chiều ngang khi số lượng cửa hàng vượt năng lực một máy chủ cơ sở dữ liệu. Hướng thứ hai là xây dựng chợ giao diện (template marketplace) trên nền cơ chế Master Template: bộ sưu tập `MasterTemplateCatalog` đã hỗ trợ cờ `isCustom`, tạo tiền đề cho phép nhà thiết kế bên thứ ba đóng góp và kinh doanh mẫu giao diện. Hướng thứ ba là khai thác dữ liệu hành vi đã được thu thập trong mô-đun tương tác (lịch sử tìm kiếm, danh sách yêu thích) để xây dựng hệ gợi ý sản phẩm cá nhân hóa cho từng cửa hàng. Cuối cùng, quy trình xử lý ảnh sản phẩm có thể được nâng cấp bằng dịch vụ tách nền tự động dựa trên học máy, chạy nền qua hàng đợi tác vụ sẵn có của hệ thống, giúp chủ cửa hàng có ảnh sản phẩm chuyên

nghiệp mà không cần công cụ chỉnh sửa bên ngoài.

TÀI LIỆU THAM KHẢO

- [1] VECOM, *Báo cáo Chỉ số Thương mại điện tử Việt Nam 2023*. **urlseen 15 january 2025. url:** <https://vecom.vn/>
- [2] Haravan, *Bảng giá các gói dịch vụ Haravan*. **urlseen 15 june 2026. url:** <https://www.haravan.com/pages/pricing>
- [3] Shopify Inc., *Shopify Pricing Plans*. **urlseen 15 january 2025. url:** <https://www.shopify.com/pricing>
- [4] Shero Commerce, *Shopify Pricing (2026): Plans, Fees & Real Costs*. **urlseen 15 june 2026. url:** <https://sherocommerce.com/blogs/insights/shopify-pricing>
- [5] Swell, *WooCommerce Pricing in 2026: Full Cost Breakdown*. **urlseen 15 june 2026. url:** <https://www.swell.is/content/woocommerce-pricing>
- [6] BuiltWith, *eCommerce Technologies Market Share*. **urlseen 15 june 2026. url:** <https://trends.builtwith.com/shop>
- [7] Red Stag Fulfillment, *Shopify Market Share Statistics 2026: Global & Regional Data*. **urlseen 15 june 2026. url:** <https://redstagfulfillment.com/shopify-market-share/>
- [8] MobiLoud, *WooCommerce vs Shopify: Market Share Statistics for 2026*. **urlseen 15 june 2026. url:** <https://www.mobiloud.com/blog/woocommerce-vs-shopify-market-share-statistics>
- [9] StoreLeads, *WooCommerce Platform Report (August 2025)*. **urlseen 15 june 2026. url:** <https://storeleads.app>
- [10] Automattic Inc., *WooCommerce Features*. **urlseen 15 january 2025. url:** <https://woocommerce.com/>
- [11] Haravan, *Nền tảng thương mại điện tử Haravan*. **urlseen 15 january 2025. url:** <https://www.haravan.com/>
- [12] Adobe Inc., *Adobe Commerce (Magento)*. **urlseen 15 january 2025. url:** <https://business.adobe.com/products/magento/magento-commerce.html>
- [13] Cục Thương mại điện tử và Kinh tế số (iDEA), Bộ Công Thương, *Sách trắng Thương mại điện tử Việt Nam*. **urlseen 15 june 2026. url:** <https://idea.gov.vn/>
- [14] VECOM, *Vietnam E-Business Index Report 2025*. **urlseen 15 june 2026. url:** <http://en.vecom.vn/vietnam-e-business-report-2025>

-
- [15] VECOM, *Báo cáo Chỉ số Thương mại điện tử Việt Nam 2024*. **urlseen** 15 **january** 2025. **url:** <https://vecom.vn/>
- [16] Vercel, *Turborepo Documentation*. **urlseen** 15 **january** 2025. **url:** <https://turbo.build/repo/docs>
- [17] Kamil Myśliwiec et al., *NestJS Documentation*. **urlseen** 15 **january** 2025. **url:** <https://docs.nestjs.com>
- [18] Vercel, *Next.js Documentation*. **urlseen** 15 **january** 2025. **url:** <https://nextjs.org/docs>
- [19] Prisma Data Inc., *Prisma Documentation*. **urlseen** 15 **january** 2025. **url:** <https://www.prisma.io/docs>
- [20] The PostgreSQL Global Development Group, *PostgreSQL 15 Documentation*. **urlseen** 15 **january** 2025. **url:** <https://www.postgresql.org/docs/15>
- [21] MongoDB Inc., *MongoDB Manual*. **urlseen** 15 **january** 2025. **url:** <https://www.mongodb.com/docs/manual>
- [22] Redis Ltd., *Redis Documentation*. **urlseen** 15 **january** 2025. **url:** <https://redis.io/docs>
- [23] MinIO Inc., *MinIO Documentation*. **urlseen** 15 **january** 2025. **url:** <https://min.io/docs/minio/linux/index.html>
- [24] Better Auth Team, *Better Auth Documentation*. **urlseen** 15 **january** 2025. **url:** <https://www.better-auth.com/docs>
- [25] The Linux Foundation, *Kubernetes Documentation*. **urlseen** 15 **january** 2025. **url:** <https://kubernetes.io/docs>
- [26] Docker Inc., *Docker Documentation*. **urlseen** 15 **january** 2025. **url:** <https://docs.docker.com>
- [27] C.-P. Bezemer and A. Zaidman, “Multi-tenant SaaS applications: maintenance dream or nightmare?” **in** *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE) 2010*, **pages** 88–92.
- [28] OpenJS Foundation, *Node.js Documentation: AsyncLocalStorage*. **urlseen** 13 **june** 2026. **url:** https://nodejs.org/api/async_context.html

PHỤ LỤC

A. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này đặc tả bổ sung các trường hợp sử dụng chưa được trình bày chi tiết trong mục 2.3 của Chương 2.

A.1 Đặc tả use case “Quản lý khuyến mãi” (UC-02)

Tên use case	Quản lý khuyến mãi
Tác nhân	Chủ cửa hàng
Mục đích	Tạo và quản lý các mã giảm giá áp dụng khi khách đặt hàng
Tiền điều kiện	Chủ cửa hàng đã đăng nhập vào phiên quản trị
Hậu điều kiện	Mã khuyến mãi được lưu với các ràng buộc hiệu lực; số lượt dùng được theo dõi tự động qua từng đơn hàng
Luồng chính	1. Chủ cửa hàng tạo chương trình khuyến mãi: mã, loại giảm (theo phần trăm hoặc số tiền cố định), giá trị giảm 2. Chủ cửa hàng đặt thời gian hiệu lực (ngày bắt đầu, ngày hết hạn) và giới hạn tổng số lượt dùng 3. Hệ thống lưu chương trình ở trạng thái kích hoạt 4. Khi khách nhập mã lúc thanh toán, hệ thống kiểm tra hiệu lực, trừ giảm giá vào tổng đơn và tăng bộ đếm lượt dùng trong cùng giao dịch với đơn hàng
Luồng phát sinh	[4a] Mã không tồn tại, chưa đến hạn, đã hết hạn hoặc bị tắt → từ chối với thông báo mã không hợp lệ [4b] Mã vượt giới hạn lượt dùng → từ chối với thông báo đã hết lượt

Bảng A.1: Đặc tả use case Quản lý khuyến mãi (UC-02)

Bảng A.1 mô tả vòng đời của một mã khuyến mãi; điểm cần lưu ý là việc ghi nhận lượt dùng diễn ra trong cùng giao dịch nguyên tử với việc tạo đơn, nên không thể xảy ra trường hợp mã bị dùng vượt giới hạn do hai đơn đặt đồng thời.

A.2 Đặc tả use case “Quản lý giỏ hàng” (UC-08)

Tên use case	Quản lý giỏ hàng
Tác nhân	Khách hàng
Mục đích	Lưu các sản phẩm khách định mua trước khi thanh toán
Tiền điều kiện	Khách hàng đã đăng nhập tài khoản mua hàng của cửa hàng
Hậu điều kiện	Giỏ hàng được lưu phía máy chủ, mỗi khách có đúng một giỏ trên mỗi cửa hàng
Luồng chính	1. Khách bấm thêm một biến thể sản phẩm vào giỏ kèm số lượng 2. Hệ thống tìm giỏ theo cặp khóa (cửa hàng, khách hàng); chưa có thì tạo mới 3. Hệ thống thêm dòng hàng hoặc cộng dồn số lượng nếu biến thể đã có trong giỏ 4. Khách xem lại giỏ, điều chỉnh số lượng hoặc xóa từng dòng 5. Khi khách đặt hàng từ giỏ, toàn bộ dòng hàng được chuyển thành đơn và giỏ được dọn sạch trong cùng giao dịch
Luồng phát sinh	[1a] Khách chưa đăng nhập → chuyển hướng sang trang đăng nhập kèm đường dẫn quay lại [3a] Biến thể không thuộc cửa hàng hiện tại → từ chối thao tác

Bảng A.2: Đặc tả use case Quản lý giỏ hàng (UC-08)

Bảng A.2 cho thấy giỏ hàng được quản lý hoàn toàn phía máy chủ và ràng buộc duy nhất theo cặp (cửa hàng, khách hàng), bảo đảm giỏ của khách tại cửa hàng này không lẫn với giỏ của chính khách đó tại cửa hàng khác.

A.3 Đặc tả use case “Ánh xạ tên miền tùy chỉnh” (UC-10)

Tên use case	Ánh xạ tên miền tùy chỉnh
Tác nhân	Chủ cửa hàng
Mục đích	Cho phép cửa hàng hoạt động dưới tên miền riêng của chủ cửa hàng
Tiền điều kiện	Cửa hàng đã được khởi tạo; chủ cửa hàng sở hữu tên miền muốn ánh xạ
Hậu điều kiện	Tên miền được ghi vào bản ghi cửa hàng với ràng buộc duy nhất toàn hệ thống; trạng thái xác minh được cập nhật; bước cuối của trình hướng dẫn khởi tạo được đánh dấu hoàn thành
Luồng chính	1. Chủ cửa hàng nhập tên miền riêng trong phần cài đặt 2. Hệ thống kiểm tra tên miền chưa thuộc cửa hàng nào khác 3. Chủ cửa hàng trả bản ghi DNS của tên miền về hệ thống theo hướng dẫn 4. Hệ thống xác minh tên miền và đánh dấu đã xác minh 5. Mọi truy cập tới tên miền này được middleware của Storefront phân giải về đúng cửa hàng
Luồng phát sinh	[2a] Tên miền đã được cửa hàng khác sử dụng → từ chối với thông báo trùng lặp [4a] DNS chưa trả đúng → trạng thái giữ nguyên chưa xác minh, chủ cửa hàng có thể thử lại

Bảng A.3: Đặc tả use case Ánh xạ tên miền tùy chỉnh (UC-10)

Bảng A.3 mô tả bước cuối trong trình hướng dẫn tám bước khởi tạo cửa hàng: sau khi tên miền được xác minh, cửa hàng chuyển sang trạng thái xuất bản và có thể truy cập công khai.